

操作系统内核技术文档

wllos

华南师范大学

吴秉谦 李骏东 罗锦程

2026 年 8 月 14 日

目录

第 1 章 摘要	1
1.1 WLL0S 内核介绍	1
1.2 内核整体设计	1
第 2 章 系统概述	2
2.1 总体架构	2
2.2 设计思路与实现重点	2
2.3 代码结构	3
2.4 启动与初始化流程	3
2.5 运行与评测路径	4
2.6 模块协同关系	5
第 3 章 内存管理模块	6
3.1 内存管理总体设计	6
3.2 物理内存与内核堆	6
3.3 页表与地址空间切换	7
3.4 用户地址空间组织	8
3.5 用户进程内存布局与程序装载	9
3.6 内存映射接口	9
3.7 懒分配与缺页处理	10
3.8 写时复制与地址空间共享	11
3.9 用户内存访问与安全保证	12
3.10 SMP 一致性与性能优化	12
3.11 本章小结	13
第 4 章 进程与调度模块	14
4.1 模块概述	14
4.2 执行实体与组织关系	17
4.3 核心数据结构与资源组织	20
4.4 进程与线程生命周期	24
4.5 调度器总体设计	29
4.6 阻塞、唤醒与等待机制	34
4.7 上下文切换与抢占机制	38
4.8 本章小结	41
第 5 章 文件系统与 I/O 模块	42
5.1 模块概述	42
5.2 总体架构与核心对象	42
5.3 路径解析、权限与挂载管理	43
5.4 命名空间事务与查找缓存	43
5.5 文件描述符与资源生命周期	44

5.6	文件读写与偏移维护	45
5.7	MemFS、ext4 与缓存同步	45
5.8	管道、等待与设备 I/O	46
5.9	测试验证与当前边界	46
5.10	本章小结	47
第 6 章	系统调用与兼容模块	48
6.1	系统调用总体设计	48
6.2	调用入口与分发流程	48
6.3	用户内存访问	49
6.4	内核对象与资源协同	49
6.5	阻塞调用与可恢复执行	50
6.6	信号、同步与时间语义	50
6.7	错误返回与兼容策略	51
6.8	跨架构 ABI 适配	51
6.9	兼容能力与扩展边界	52
6.10	本章小结	52
第 7 章	总结与展望	53
7.1	项目总体总结	53
7.2	初赛与决赛阶段总结	53
7.3	项目亮点与核心优势	54
第 8 章	AI 辅助使用说明	56
8.1	使用原则	56
8.2	使用场景与验证方式	56
8.3	工程边界与责任	57

第 1 章 摘要

1.1 WLLOS 内核介绍

wllos 是一个使用 Rust 编写的 `no_std` 类 Unix 宏内核，面向 RISC-V64 与 LoongArch64 两种体系结构。项目以 Linux 用户态兼容为目标，可装载静态或动态链接的 ELF 程序，并提供进程、虚拟内存、文件系统、信号、同步、时间和基础网络接口。当前主要运行环境是 QEMU；内核同时建立了 VisionFive 2 和 Loongson 2K1000LA 的平台识别边界。截至到 8 月 17 日前，我们暂时还没有完成实板能力的验证。

项目面向操作系统内核比赛中的真实工作负载，而不是只运行少量自制应用。用户空间可以运行 BusyBox、Lua、libc 测试和多种性能程序，BuildStorm 场景还会同时使用动态链接、进程创建、并行编译、文件系统和内存映射。内核因此需要在 Linux ABI、资源生命周期和多核并发方面形成完整闭环。

1.2 内核整体设计

wllos 采用“共享策略、平台机制分离”的分层宏内核设计。进程、调度、内存、文件系统和 Linux ABI 位于架构无关层，页表格式、Trap 返回、中断、时钟、CPU 启动和设备接入位于 PolyHAL 与平台边界。两种架构共用同一套资源语义，只在必须接触硬件的位置进行适配。

内核使用明确的所有权和状态转换组织资源。任务对象维护线程与进程关系，地址空间对象协调虚拟区域、驻留页面和页表，文件描述符引用统一的打开文件对象，等待队列连接阻塞条件与唤醒事件。页帧、文件、任务和地址空间均按照引用生命周期回收，减少跨模块手工释放造成的不一致。

多核设计不是对单核调度的简单复制。系统从设备树获得 CPU 与内存事实，只有完成本地 Trap、时钟、跨核通知和调度初始化的 CPU 才进入在线集合。任务发布、空闲 CPU 领取、跨核唤醒和 TLB 失效采用明确协议，使调度、地址空间和 I/O 可以在多个 CPU 上共同运行。

第 2 章 系统概述

2.1 总体架构

wll0s 从上到下分为用户程序、系统调用边界、内核核心、硬件抽象和硬件平台五层。用户程序只依赖 Linux ABI；系统调用层负责参数检查、用户数据复制和请求分发；内核核心实现进程、内存、文件系统、IPC 与时间语义；硬件抽象层统一页表、Trap、时钟、中断和设备操作；底层由 RISC-V64、LoongArch64 平台提供实际执行环境。

用户程序层	glibc、musl、BusyBox、编译工具链和评测程序
系统调用层	Linux ABI、用户数据复制、错误语义和请求分发
内核核心层	任务调度、虚拟内存、VFS、文件描述符、信号、IPC 和时间
硬件抽象层	页表、Trap、TLB、中断、时钟、CPU 启动和 VirtIO 接入
硬件平台层	RISC-V64 QEMU、LoongArch64 QEMU 及已识别的目标板平台

图 2-1 系统总体架构

各层之间按照语义而非源码文件相互依赖。系统调用不直接操作设备，文件系统不直接解释用户指针，任务调度不编码具体中断寄存器，地址空间也不直接决定体系结构页表位。模块之间通过稳定对象和状态协议连接，使同一套内核主体能够在两种架构上保持一致行为。

2.2 设计思路与实现重点

wll0s 以真实用户程序为牵引，但实现顺序仍以长期架构为约束。一个评测失败需要追溯到任务、地址空间、文件对象或等待关系，而不是增加只对某个程序生效的分支。功能补齐与性能优化都必须保持 Linux 语义、双架构一致性和资源生命周期不变。

当前实现重点集中在五个方面：稳定的资源所有权、平台机制与公共策略分离、SMP 下的发布与失效协议、面向真实负载的性能优化，以及可复现的证据链。内存侧将逻辑区域与驻留页面分开，调度侧消除丢失唤醒窗口，文件系统复用路径查找结果并使用有界缓存，内核堆通过每 CPU 小对象缓存降低中央锁竞争。

设计原则	架构落点	作用
资源生命周期明确	任务、页帧、地址空间和文件对象使用统一所有权关系。	便于检查共享、退出与错误回滚。
平台机制分离	架构差异集中在 PolyHAL 与平台层。	两种架构共用上层语义，减少重复实现。
并发协议先行	任务发布、阻塞唤醒、页表更新和缓存失效规定固定顺序。	避免多核下的丢失唤醒和过期映射。
有界性能优化	每 CPU 缓存、缺页预取和 VFS 缓存均设置容量与回退。	降低热点开销，同时控制内存占用。
证据可复现	保存构建环境、镜像与内核哈希、串口日志和评测摘要。	使功能与性能结论可以复查。

2.3 代码结构

项目按照内核职责划分源码，同时将架构补丁、构建脚本和评测证据与内核主体分开保存。理解系统时可以从启动入口进入，再沿平台、任务、内存、文件系统和系统调用模块展开，无需依赖完整文件树。

路径	作用
<code>os/src/main.rs</code>	启动 CPU 与次级 CPU 的初始化入口，组织内存、平台、设备、文件系统和调度启动。
<code>os/src/platform.rs</code> 、 <code>config</code>	保存设备树发现的平台事实、CPU 状态、地址布局和架构配置。
<code>os/src/task</code>	进程线程模型、调度队列、阻塞唤醒、信号状态和运行入口。
<code>os/src/mm</code>	物理页、内核堆、页表、地址空间、程序装载和内存映射。
<code>os/src/fs</code> 、 <code>drivers</code>	VFS、MemFS、ext4、文件描述符、管道、伪设备及 VirtIO 块设备。
<code>os/src/syscall</code> 、 <code>trap</code>	Linux ABI 分发、用户态出入、异常处理和系统调用返回。
<code>patch/polyhal</code>	RISC-V64 与 LoongArch64 的页表、Trap、启动和硬件机制适配。
<code>scripts</code> 、 <code>testdata</code> 、 <code>docs</code>	构建运行、日志解析、回归证据和设计说明。

2.4 启动与初始化流程

启动流程由启动 CPU 负责建立全局资源，再释放次级 CPU 加入调度。启动入口首先解析设备树，取得物理内存、CPU 和平台类型；随后建立基础分配器、页帧引用信息、动态内核堆和内核页表。Trap、时钟和本地跨核通知就绪后，启动 CPU 才标记为在线。

阶段	主要对象	说明
平台发现	设备树、CPU 与内存事实	确认平台类型、CPU 硬件编号和可用物理内存。
内存基础	页帧、内核堆、内核页表	建立后续动态对象与用户地址空间所需的分配能力。
本地执行环境	Trap、时钟、中断与跨核通知	使当前 CPU 能处理中断、异常和调度事件。
设备与文件系统	VirtIO、ext4、MemFS 与 VFS	挂载运行时根文件系统，提供程序和动态库。
多核上线	次级启动栈、在线集合和调度现场	逐个启动次级 CPU，失败 CPU 不进入可调度集合。
用户态运行	初始进程、地址空间和调度器	装载用户程序并进入多核任务调度。

次级 CPU 使用独立启动栈等待全局资源发布，随后恢复内核页表，初始化本地 Trap、时钟和跨核通知，最后进入在线状态并运行次级调度循环。在线集合是任务亲和性、空闲 CPU 发布和 TLB 跨核失效的共同依据，因此“硬件存在”与“可参与调度”被明确区分。

2.5 运行与评测路径

运行路径以外部根文件系统中的真实程序为准。构建阶段生成对应架构的内核镜像，QEMU 加载官方磁盘镜像并提供多核、内存和 VirtIO 设备；内核挂载 ext4 后启动镜像中的入口程序或评测脚本。用户程序通过标准系统调用驱动进程、内存和文件系统，最终结果经串口输出交给评测器解析。

构建	固定工具链与架构目标	生成 RISC-V64 或 LoongArch64 内核，并记录构建配置。
启动	QEMU、官方镜像与 VirtIO	按规定 CPU、内存和块设备参数启动真实运行环境。
执行	用户程序与系统调用	动态链接、编译、文件访问和进程并发共同驱动内核。
采集	串口、状态标记和时间数据	保存完整运行过程，不以包装脚本退出码代替用户态结果。
判定	评测器与证据索引	结合成功标记、日志、哈希和摘要形成可复现结论。

图 2-2 构建、运行与证据链

当前双架构公开流程均能完成完整编译成功标记，官方自动评测项解析为 180/180。该结论对应公开镜像、规定资源和保存的运行证据；人工设计文档分值、正式比赛环境的时间分和隐藏环境结果仍以评委实际复核为准。内核不会根据测试路径、命令名称或预期输出改变正常行为。

2.6 模块协同关系

wll0s 的模块边界用于划分责任，而不是割裂执行流程。一次系统调用从 **Trap** 进入内核后，可能同时访问任务状态、地址空间、文件描述符和等待队列。协作过程由资源对象承载：上层提出语义请求，下层完成状态变更，错误沿统一边界返回，任何失败都不能留下部分提交的资源。

多核环境进一步要求模块共享相同的发布顺序。任务变为可运行后才通知空闲 CPU，页表更新并完成 TLB 失效后才释放旧页帧，文件写回收集数据后才离开地址空间锁执行 I/O。统一顺序让调度、内存和文件系统在压力场景下仍保持可解释状态。

关键流程	参与模块	协作重点
程序映像替换	VFS、程序装载、地址空间、任务与 Trap	先完整构造新映像，再一次性替换旧执行环境。
进程创建与等待	任务、写时复制、文件表、信号与等待队列	按共享标志复制资源，并在退出时发布可观察结果。
缺页与文件映射	Trap 、地址空间、页帧、文件系统与 TLB	根据区域语义装入页面，并同步所有相关 CPU。
阻塞 I/O	文件对象、等待队列、调度器与信号	先登记等待条件，再让出 CPU，事件到达后精确唤醒。
共享文件写回	地址空间、页缓存、VFS 与 ext4	分离元数据锁和磁盘 I/O，保持页面与文件状态一致。
跨核任务发布	就绪队列、空闲集合、CPU 亲和性与 IPI	发布任务后领取目标 CPU，避免重复通知和丢失唤醒。

第 3 章 内存管理模块

3.1 内存管理总体设计

wll0s 的内存管理模块连接硬件页表、进程管理与系统调用三个层次。它既要管理内核自身使用的物理页和动态对象，也要为每个用户进程维护相互隔离的虚拟地址空间。设计目标可以概括为：保证地址隔离和访问权限正确，支持真实用户程序装载，提供 Linux 常用映射语义，并在多核环境中维持页表与处理器缓存的一致性。

模块采用分层设计。物理内存层提供页帧和内核堆；页表层负责虚拟地址到物理地址的硬件映射；地址空间层记录区域权限、数据来源和驻留页面；系统调用与异常处理层把进程堆、文件映射、共享内存和缺页请求转换为统一的地址空间操作。上层只描述内存语义，体系结构差异集中在页表与刷新接口中。

接口与异常层	进程堆、内存映射、共享内存、程序装载和缺页异常
地址空间层	虚拟内存区域、映射后端、驻留页状态和生命周期
页表抽象层	地址转换、权限编码、地址空间切换和 TLB 同步
基础分配层	物理页帧、引用计数、内核堆和每 CPU 缓存

图 3-1 内存管理分层架构

内存管理内部按照职责划分为页帧分配、堆分配、页表封装、虚拟内存区域、进程地址空间和程序装载等部分。任务模块持有进程地址空间，Trap 模块把缺页原因交给地址空间处理，文件系统为文件映射提供数据和写回能力，硬件抽象层负责发布页表并完成本地或跨核失效。各部分通过少量稳定接口协作，避免上层直接修改页表细节。

系统启动后，内核首先根据平台提供的内存区域建立页帧分配器，同时排除内核镜像和保留区域。随后初始化页帧引用信息和内核动态堆，并接管启动阶段已经建立的内核页表。完成这些步骤后，文件系统、任务对象和用户页表才能安全地申请动态内存。该顺序保证任何被发布到页表中的物理页都已有明确所有者。

3.2 物理内存与内核堆

不同平台提供的物理内存起止位置并不完全相同，因此内核以可用区间为单位登记物理内存，而不假设一段固定连续空间。区间边界先按页对齐，再交给统一分配器管理。内核同时维护总页数和空闲页数，用于判断内存压力，并避免把设备地址或保留页误当作普通内存分配。

物理页帧采用伙伴系统管理。单页用于用户数据页和页表，多页连续区域可用于具有连续性要求的内核对象。伙伴系统能够快速拆分和合并空闲块，在分配速度与外部碎片之间取得平衡。分配得到的页面会先清零，防止新进程读取到先前使用者遗留的数据。

一个物理页可能被父子进程、共享内存或文件页缓存同时引用。为此，wll0s 为受管理页帧维护原子引用计数，并用页帧追踪对象表达所有权。复制追踪对象表示增加一个

合法引用，转移追踪对象表示所有权从一个内核结构移动到另一个结构。这样可以统一描述私有页、共享页和写时复制页。

页帧追踪对象离开作用域时会自动减少引用计数；计数归零后，物理页才返回伙伴系统。页表页、用户数据页和连续页采用明确的所有权转换，避免重复释放或仍在使用时提前释放。该机制利用 Rust 的生命周期管理减少手工回收路径，同时保留运行期一致性检查。

内核堆负责分配任务对象、文件节点、映射元数据和缓存条目等大小不一的对象。系统先使用启动阶段预留的堆空间，在需要时再从空闲物理内存扩展。中央堆仍采用伙伴分配思想，使大对象和非常规对齐请求能够统一处理；扩展上限同时受物理内存规模约束，避免堆过度占用用户页帧。

多核运行时，所有小对象都访问同一个中央堆会形成锁竞争。wlllos 按 CPU 设置小对象缓存，并按二次幂大小分类。常见分配和释放优先在当前 CPU 的缓存中完成，缓存不足或过满时再批量与中央堆交换。该设计缩短了全局锁持有时间，也减少了频繁申请任务和文件系统对象时的同步开销。

3.3 页表与地址空间切换

wlllos 通过硬件抽象层封装 RISC-V64 与 LoongArch64 的页表差异。内存管理只使用统一的映射、解除映射、地址转换和根页表切换语义，具体页表级数、控制寄存器及刷新指令由架构层完成。这样既保持上层地址空间模型一致，也允许各架构选择符合硬件特性的实现。

用户虚拟地址按页号和页内偏移拆分，页表项保存对应物理页号以及读、写、执行和用户访问权限。建立映射前，内核会检查地址范围、页对齐和权限组合。逻辑权限还要经过架构适配后才能写入硬件页表，例如写权限在部分架构上必须与读权限同时存在。

内核拥有稳定的内核页表，用于访问内核代码、数据和设备区域；每个用户进程拥有独立的用户页表根。进入用户态前激活目标进程的页表，陷入内核或切换任务时恢复合适的内核执行环境。用户地址不能越过平台规定的上限，内核地址也不会以用户权限暴露，从页表层建立基本隔离边界。

频繁切换页表后完全清空 TLB 会损失已有地址转换缓存。wlllos 为用户地址空间分配非零标识，使处理器能够区分来自不同页表的缓存项。标识释放后先进入退役集合，只有完成全核刷新后才能重新使用；当硬件标识不足时则退回保守模式，以正确性优先。

调度器准备运行用户任务时，会先公布当前 CPU 即将使用的页表根，再切换硬件页表。若目标页表及标识已经处于活动状态，可跳过重复切换；否则发布新的根页表并按架构要求完成本地同步。公布状态早于进入用户态，使其他 CPU 修改共享地址空间时能够准确识别需要通知的处理器。

页表项被撤销或降低权限后，旧的 TLB 项可能继续允许访问。wlllos 先修改页表，再向正在使用同一地址空间的 CPU 发出失效请求，最后才释放旧页帧。RISC-V64 可以结合地址空间标识保留无关缓存，LoongArch64 当前采用更保守的本地刷新策略。两种路径共享相同的安全顺序。

3.4 用户地址空间组织

每个进程地址空间由根页表、地址空间标识和一组有序虚拟内存区域组成。根页表表示处理器当前可见的映射，区域集合表示内核理解的长期语义。线程可以共享同一个地址空间对象，普通进程创建则得到新的页表与区域视图。硬件状态与软件描述始终由同一地址空间统一协调。

虚拟内存区域描述一段连续地址范围及其读、写、执行权限，作用类似 Linux 的 VMA。区域之间保持有序且不重叠，因此可以进行范围覆盖检查、空闲区查找和缺页归属判断。没有叶子页表项的保留区域也会保留在区域集合中，使后续权限调整和按需分配仍有可靠依据。

区域的数据来源分为匿名内存、文件和共享内存。匿名内存初始内容为零，常用于堆、栈和临时缓冲；文件映射记录文件身份、偏移和共享属性；共享内存记录共享段身份及段内偏移。统一的后端描述使缺页处理能够根据区域语义选择清零、读文件或复用共享页。

权限修改或部分解除映射可能只影响一个区域的中间部分，因此系统会在边界处拆分区域，再处理目标区间。操作结束后，地址相邻、权限相同且后端连续的区域可以重新合并。拆分时同步调整文件偏移和驻留页相对位置，合并时保持页面所有权不变，从而防止区域元数据与实际页面错位。

逻辑区域不再要求范围内所有页面都已经分配。实际存在的页面由独立驻留集合管理，并按相对于区域起点的页号索引。连续驻留页聚合为小段，空洞无需占用页帧或逐页元数据。缺页只插入新页面，不改变逻辑区域拓扑，因而稀疏匿名映射和大文件映射的维护成本更低。

驻留页除物理页引用外，还记录私有、写时复制、共享、文件干净和文件脏等状态。这些状态表达实际页面当前如何被使用，而区域权限只表达用户期望的访问策略。两者结合后才能得到最终页表权限，例如逻辑上可写但处于写时复制状态的页面必须暂时以只读方式发布。

页面状态	含义	典型场景
私有	仅由当前地址空间拥有	新分配的堆、栈和匿名页。
写时复制	多个地址空间共享，只读发布	进程创建后尚未被写入的私有页。
共享	多个映射观察同一物理内容	共享内存与共享文件映射。
文件干净	内容与文件一致	只读文件页和私有文件页初次装入。
文件脏	内存内容已被修改	需要同步或解除映射时写回的共享文件页。

将区域语义与驻留状态分开，是当前内存架构的关键改进。区域拆分、合并和权限修改只处理地址范围与后端策略；按需分配、文件预读和写时复制只改变驻留页面。这样可以避免一次单页缺页造成大量区域碎片，也让共享、写回和页面回收的责任边界更加清晰。

3.5 用户进程内存布局与程序装载

用户地址空间从低地址到高地址主要包含程序段、进程堆、内存映射区和用户栈。系统为普通映射设置低、高两个候选区间，在低地址空间压力较大时可以转向高地址区，减少大型用户程序反复申请失败。用户栈位于平台规定的高地址附近，并与其他区域保持不重叠。

程序装载首先检查 ELF 魔数、位宽、目标架构和程序头边界，只接受与当前平台匹配的 64 位可执行文件或共享对象。解析阶段只建立经过校验的段描述，发现越界、溢出或不合理对齐立即终止装载。由此把不可信文件内容限制在进入地址空间之前。

每个可装载段按照页边界扩展为虚拟内存区域，文件内容复制到对应位置，文件未覆盖而内存需要的部分保持为零。段标志被转换为读、写和执行权限，并附加用户访问属性。不同段发生非法重叠时拒绝装载，避免可写区域意外覆盖代码或页表中出现含义冲突的权限。

位置无关程序需要选择不会与现有区域冲突的装载偏移。若 ELF 指定动态解释器，内核同时装载解释器，并把原程序入口和程序头信息交给它；用户态最终从解释器入口开始执行。该设计兼容静态程序、位置无关可执行文件以及 glibc、musl 使用的动态链接流程。

内核在用户栈范围建立可读写映射，并从栈顶向低地址写入字符串、指针数组和启动信息。构造过程保证指针宽度和栈对齐满足目标架构 ABI，同时检查所有写入均未越过栈底。完成后，栈指针直接指向用户入口约定的启动布局。

用户栈包含参数数量、参数数组、环境变量数组和辅助向量。辅助向量向 C 运行库及动态链接器提供程序入口、程序头位置、页大小、用户身份等信息。信号返回所需的用户态跳板也在地址空间建立。内核只提供启动所需事实，不在内核态承担用户动态链接逻辑。

执行新程序时，系统先在独立地址空间中完成解析、段映射和用户栈构造。只有新映像全部成功后，才替换当前进程的旧地址空间和用户寄存器现场。失败时旧程序仍可继续处理错误；成功后再释放旧映射、重置进程堆和映射游标。这种先构造后提交的方式避免进程停留在半更新状态。

3.6 内存映射接口

进程堆以 ELF 装载结束位置为起点，分别记录用户可见的逻辑边界和实际映射到页边界后的范围。增长时只补充新增页面，收缩时释放不再覆盖的完整页面。查询请求返回当前边界，非法请求保持原值，从而符合 C 运行库对 Linux 原始 `brk` 语义的预期。

非固定映射优先参考调用者提示和进程映射游标，再在允许的低、高地址区间查找足够大的空洞。固定映射要求地址页对齐，并按标志决定拒绝重叠还是替换原区域。所有长度计算使用溢出检查，结果不得覆盖用户栈、程序保留区或超出用户地址上限。

匿名映射只建立区域语义，页面可在首次访问时分配并清零，适合大块稀疏内存。文件映射额外保存文件身份和偏移，缺页时读取对应文件页。两类映射共享相同的范围、权限和解除映射框架，因此动态链接器、线程栈和应用缓冲区不需要不同的地址空间模型。

私有映射允许进程修改自己的页面而不直接改变其他映射，进程创建后可通过写时复制延迟复制。共享映射则让多个地址空间观察同一物理内容，并在需要时把修改同步到文件。映射类型在区域建立时确定，后续拆分和合并都必须保持这一语义。

权限修改先验证目标范围已被现有区域完整覆盖，再沿边界拆分并更新目标区域。已经驻留的页面按新权限重新发布；完全无访问权限的区域可以保留逻辑描述而撤销叶子映射。降低权限后必须使旧 TLB 项失效，写时复制页则继续保持硬件只读，不能因逻辑写权限恢复而绕过复制。

解除映射可能删除完整区域，也可能保留左右两侧。系统先收集需要写回的共享文件内容，再撤销目标页表项并完成跨核失效，最后释放对应驻留页。区域的文件偏移和页面索引随拆分同步调整，使剩余映射仍指向正确的数据位置。

共享可写文件映射的修改需要回到原文件。同步操作按照映射区间计算文件偏移，收集驻留页面内容并交给文件系统写回；解除映射和固定替换也会执行相同的持久化检查。I/O 与地址空间元数据更新分阶段进行，避免长时间持有地址空间锁，并在写回失败时保留可报告的错误状态。

System V 共享内存以全局共享段保存物理页集合、权限、创建者和挂接计数。进程挂接时把同一组页帧映射到选定地址，分离时只删除当前进程的映射。删除请求先标记共享段，待最后一次挂接解除后再回收页面，从而同时满足跨进程可见性和延迟销毁语义。

3.7 懒分配与缺页处理

大型匿名映射和程序段并不一定会被全部访问，建立映射时立即分配所有物理页会增加启动延迟并浪费内存。wll0s 因此允许先记录虚拟区域和权限，在首次访问时再提供页面。逻辑地址空间规模与实际物理占用由此解耦，程序只为真正使用的部分付出分配成本。

处理器报告缺页后，Trap 层提供故障地址以及读取、写入或取指类型。地址空间首先查找覆盖该地址的区域，检查权限，再判断页面属于写时复制、已经驻留但尚未发布，还是尚未分配。处理成功后建立或更新页表项并返回原指令重试；无法解释的访问转化为进程可见错误。

判断阶段	条件	处理结果
区域与权限	无覆盖区域或访问类型不允许	拒绝访问并交由异常退出路径处理。
写时复制	私有共享页发生写入	复用或复制物理页，恢复当前映射写权限。
已有驻留页	页面存在但页表项尚未发布	按区域策略和页面状态重新建立映射。
匿名后端	页面尚未分配	分配清零页并建立映射。
文件后端	页面尚未装入	读取文件或复用干净页缓存后建立映射。

图 3-2 缺页处理决策流程

取指要求区域可执行，写入要求区域可写，读取要求区域具备可读语义。检查使用逻辑区域权限，而不是只查看当前页表项，因为写时复制会主动移除硬件写位，保留区域也可能暂时没有页表项。先判断策略再处理页面状态，可以区分合法的延迟映射与真正的越权访问。

匿名缺页从页帧分配器取得清零页面，并加入对应区域的驻留集合。为降低连续访问造成的频繁异常，系统会在故障页之后进行有界批量分配，但遇到已有驻留页、区域边界或内存压力立即停止。批量策略只改变驻留状态，不拆分逻辑区域。

文件映射缺页根据区域起点、映射偏移和故障页位置计算文件偏移。读取不足一页的尾部保持为零，超出合法映射范围的访问被拒绝。私有文件页可以进入写时复制路径，共享文件页则保留与文件的同步关系，使同一套缺页流程兼容程序装载和普通文件映射。

对于只读且适合缓存的普通文件，内核按文件身份和页偏移复用干净物理页，避免多个进程重复读取相同代码或只读数据。一次故障还可以触发有界预读，并把连续页面批量加入驻留集合。任何页面被写入后都不能继续作为干净缓存共享，以保持文件内容和页面状态一致。

缺页处理遵循“先获得页面所有权，再发布页表项”的顺序。若分配、文件读取或元数据插入失败，候选页由局部所有权自动回收，原映射保持不变。已经驻留但页表项缺失时可以直接恢复映射；处理完成后按需要执行 TLB 同步，避免返回用户态后继续命中过期限。

3.8 写时复制与地址空间共享

直接复制整个用户地址空间实现简单，但大量页面在子进程立即执行新程序时从未被修改。wlos 按区域语义选择继承策略：私有可写页采用写时复制，只读页和显式共享页继续共享，确实不能共享的页面才立即复制。创建成本因此主要来自新页表和少量元数据。

建立写时复制时，父子地址空间各自持有同一物理页的引用，驻留状态改为写时复制，双方页表均以只读方式发布。逻辑区域仍保留可写策略，以便区分正常只读页面和等待复制的页面。父页表权限改变后完成跨核失效，才能让父进程继续运行。

写入写时复制页会触发权限异常。内核定位驻留页后，申请新页并复制原页内容，再把当前地址空间的页面引用替换为新页，状态恢复为私有，最后发布可写页表项。另一地址空间仍持有原页，因而父子进程从第一次写入开始看到彼此独立的内容。

如果发生写异常时原页只剩当前地址空间一个引用，就不需要复制数据。内核可以直接把页面状态改回私有并恢复写权限。引用计数既负责页面生命周期，也为这一快速路径提供判断依据，减少子进程退出后父进程再次写入页面的额外开销。

真正只读的页面可以安全地由父子进程共同使用，无需标记为写时复制。共享文件映射和共享内存则必须保持共享页面状态，即使区域具有写权限也不能在写异常时私有化。通过区分逻辑权限、映射类型和页级状态，进程创建不会破坏原有共享语义。

创建线程并请求共享虚拟内存时，多个任务引用同一个地址空间对象和同一组页表。任一线程建立映射或写入页面，其他线程立即看到结果。地址空间内部使用互斥保护区域和驻留页元数据，跨 CPU 的页表修改还要配合 TLB 通知，保证共享并不意味着无同步访问。

进程创建产生新的地址空间或共享引用，程序映像替换提交新的地址空间，线程退出减少共享引用，进程退出则解除共享内存挂接并释放最后的页表与驻留页。释放过程中先切回安全页表，再撤销映射并完成失效，最后依靠引用计数回收页帧，避免 CPU 继续访问已经重新分配的物理页。

3.9 用户内存访问与安全保证

系统调用使用用户指针前，会检查起止地址是否溢出、是否处于用户可访问范围，以及整个区间是否由具有相应权限的区域覆盖。空指针、跨越地址上限和落入区域空洞的访问统一返回错误。检查以区间为单位，不能只验证首地址。

字符串、结构体和 I/O 缓冲区可能跨越多个页面。内核按页分段完成地址转换和复制，每一段都独立检查权限及驻留状态。这样即使前一页合法、后一页未映射，也不会越过边界直接解引用；部分复制是否允许则由具体系统调用语义决定。

在实际复制前，内存管理会为读取方向确保源页面可读，为写入方向确保目标页面可写。准备过程可以补齐合法的懒分配页面，并在内核写入用户缓冲区时主动解决写时复制。系统调用层因此不需要重复理解匿名页、文件页和写时复制的区别。

内核不会假设任意时刻都能直接解引用用户虚拟地址。访问用户数据时明确使用当前任务的地址空间，完成后恢复内核执行所需的页表上下文。任务切换、异常嵌套和多核执行都遵守同一规则，防止内核把一个进程的用户地址错误地解释为另一个进程的映射。

地址错误、权限不足、内存不足和文件读取失败需要区分处理。用户传入无效地址通常转换为 `EFAULT`，非法参数返回 `EINVAL`，无法取得物理页返回 `ENOMEM`。硬件缺页若不能由合法区域解释，则进入进程异常路径。清晰的错误边界便于 C 运行库和应用程序采取正确恢复策略。

Rust 的所有权系统用于约束页帧、区域和地址空间对象的生命周期，页帧追踪对象负责自动释放共享物理页。页表操作仍涉及底层地址和必要的 `unsafe`，因此内核把这些代码集中在少量边界内，并在边界外使用受检查的对象。语言安全与运行期引用计数共同降低悬空引用和重复释放风险。

3.10 SMP 一致性与性能优化

多线程可能在不同 CPU 上同时使用同一地址空间。区域拓扑、驻留集合和页表更新由同一地址空间锁协调，读取故障、解除映射和权限修改不会交叉产生半完成状态。锁只保护内存元数据，不应覆盖可能长时间阻塞的文件 I/O，以减少共享地址空间上的锁队列。

多核页表更新遵循固定顺序：准备新元数据或新页面，修改页表项，通知相关 CPU 失效旧 TLB，确认旧映射不可再使用，最后释放被替换的页帧。映射建立时则先确保页面和元数据有效，再发布页表项。该顺序是解除映射、权限降低和写时复制都必须遵守的安全约束。

地址空间标识减少任务切换时的全量 TLB 刷新，但标识数量有限。内核优先分配未使用标识，地址空间销毁后仅将标识标记为退役；当需要复用时执行一次全核刷新，再把退役标识放回可用集合。批量回收避免每次进程退出都触发昂贵的全局同步。

地址区域保持有序，并在局部变更后只检查邻近区域是否可以合并。驻留页使用稀疏连续段保存，按需缺页不会增加区域数量。两者分工后，映射和权限操作面向少量逻辑区间，页面故障面向局部驻留状态，避免对整个地址空间反复拆分或扫描。

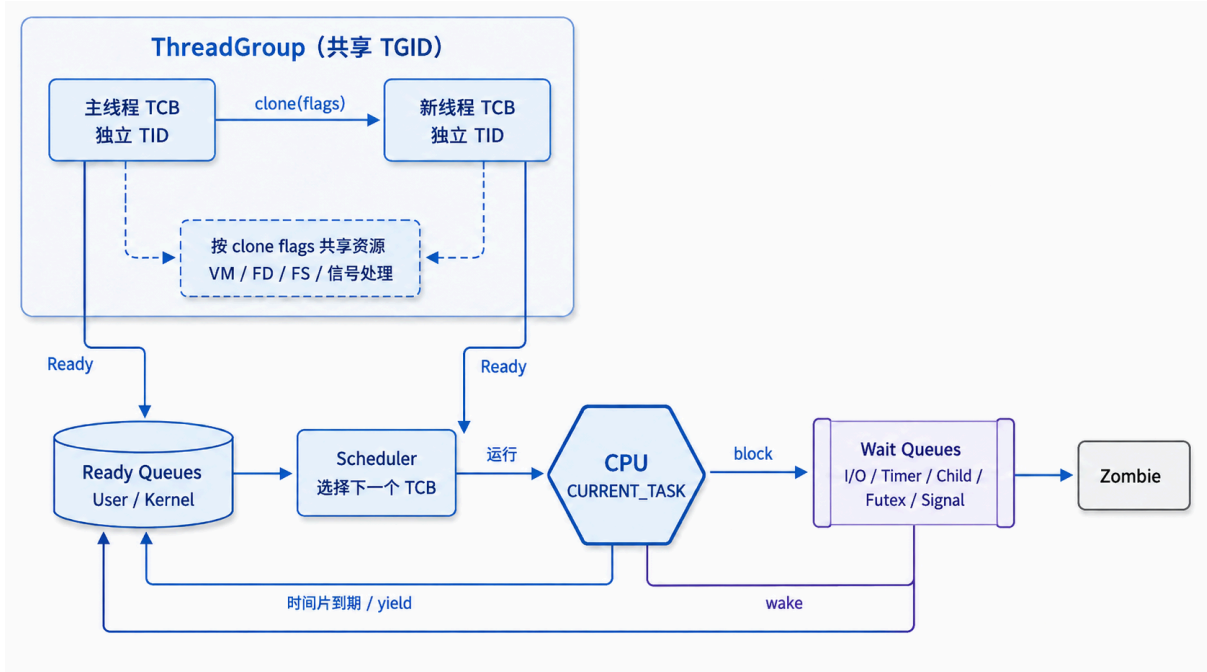
连续匿名访问采用小窗口批量分配，连续只读文件访问采用页缓存和有界预读。窗口大小受到区域边界、已有页面和可用内存限制，不能因一次故障无界占用资源。优化仍以单页正确性为基础，即使批量工作部分失败，故障页也可以独立完成或返回明确错误。

内核堆的每 CPU 小对象缓存把高频分配从中央锁中移出，地址空间标识减少切换刷新，驻留页分段降低元数据操作量，缺页批处理减少异常次数。这些优化分别作用于分配、切换、查询和装入路径，均保留中央结构作为回退，避免为了性能引入不同的资源语义。

3.11 本章小结

wlllos 以内核堆和页帧分配为基础，通过页表抽象、虚拟内存区域和驻留页状态构成统一的进程地址空间。匿名内存、文件映射和共享内存使用相同的范围管理框架，懒分配与写时复制按页补充实际资源；地址空间标识、跨核 TLB 同步和每 CPU 缓存进一步适配多核运行。该设计将硬件映射、软件语义与页面所有权分开维护，在保持 Linux 用户态兼容能力的同时，使内存生命周期和并发边界更容易检查。

第 4 章 进程与调度模块



Wll0s 的进程与调度模块负责执行实体的创建、组织、运行和回收。本章首先说明模块的职责、SMP 执行模型、Linux/POSIX 兼容目标和源码组成，再介绍任务模型、生命周期、阻塞唤醒及评测调度等具体实现。上图保留了任务、线程组、就绪队列和等待队列之间的基本关系，其中单个 CPU/CURRENT_TASK 方框表示一个 CPU 的局部视角；在当前多核实现中，每个在线 CPU 都拥有自己的当前任务槽和调度现场，并共同从全局就绪队列中选择满足亲和性的任务。

4.1 模块概述

4.1.1 模块职责与设计目标

进程与调度模块位于系统调用语义和体系结构执行机制之间。向上，它为 `clone`、`execve`、`exit`、`wait4`、`sched_yield` 和 `futex` 等系统调用提供任务对象与状态转换；向下，它调用 `trap`、页表和定时器机制完成用户态恢复、地址空间切换与时钟抢占；横向则与文件描述符表、文件系统上下文和信号状态共同维护一个进程的运行环境。

Wll0s 以任务控制块 (TaskControlBlock, TCB) 统一表示可执行实体。每个用户线程或内核任务对应一个 TCB，调度器只面向 task 进行选择 and 切换；多个用户线程再通过 ThreadGroup 组成用户可见的进程。这样既能保持调度核心的统一，也能表达 Linux 中线程私有状态与进程共享资源之间的区别。

该模块的主要设计目标如下：

设计目标	核心机制	作用
统一执行实体	<code>TaskControlBlock</code> 与 <code>ThreadGroup</code>	用同一调度框架管理用户线程、进程主线程和内核任务。
保持 Linux 进程语义	TID、TGID、PGID、SID 与父子关系	支撑线程组、进程组、会话、等待和孤儿进程收养。
保证状态转换可靠	双就绪队列、等待队列与 wait token	防止任务重复入队、丢失唤醒以及过期事件误唤醒。
支持 SMP 执行	CPU-local 当前任务、运行所有权与定向 IPI	允许任务在多个在线 CPU 上调度，同时保证同一执行现场不会被两个 CPU 并发恢复。
隔离体系结构差异	<code>TrapFrame</code> 、 <code>TaskContext</code> 与架构 trap 入口	让 RISC-V64 与 LoongArch64 复用相同的任务生命周期和调度语义。

4.1.2 SMP 多任务执行模型

WLIOS 当前采用共享就绪队列上的 SMP 多任务模型。系统分别维护用户任务就绪队列与内核任务就绪队列，两条队列仍是全局共享对象；每个在线 CPU 则拥有 CPU-local 的 `CURRENT_TASK`、`Processor` 和调度器 `TaskContext` 槽。这样的结构没有把任务预先固定到某一条 per-CPU run queue，而是在出队时按照任务状态、CPU affinity 和有效优先级选择当前 CPU 可以运行的 TCB。

每个就绪队列由 `VecDeque` 保存可运行次序，同时使用 TID 集合镜像队列成员，避免定时器、信号或 I/O 多次唤醒时将同一个任务重复加入队列。任务真正进入 `Running` 前还要原子取得 `running_cpu` 所有权；如果另一个 CPU 已经拥有该任务的用户现场，本次选择会失败并继续寻找其他候选。任务退出用户运行边界或内核任务切回调度器后再释放该所有权，因此同一 `TrapFrame` 或内核栈不会被两个 CPU 同时恢复。

普通任务按照就绪队列顺序选取。任务运行一个时间片、主动调用 `sched_yield`，或者从一次用户态 trap 返回后，如果仍处于可运行状态，就会通过本地重排队路径放回队尾，因此形成多 CPU 上的简化轮转执行。对于设置为 `SCHED_FIFO` 或 `SCHED_RR` 的任务，调度器根据实时优先级选择候选任务，并通过有限的低优先级运行预算避免次高优先级任务长期得不到执行。该实现提供了基础实时优先级选择，但尚不等同于完整的 Linux FIFO/RR 时间片与抢占规则。

用户任务与内核任务使用不同的运行现场。用户任务通过 `TrapFrame` 保存用户寄存器，在运行前激活所属 `MemorySet`，并在系统调用、中断或异常发生后回到 trap 处理路径；内核任务拥有独立内核栈，通过 `TaskContext` 保存返回地址、栈指针和被调用者保存寄存器。每个 CPU 都有独立的调度器现场，调度核心根据任务类型选择恢复方式，并通过平台层维护在线 CPU、idle mask、IPI 和地址空间状态。

调度事件	状态变化	处理方式
时间片结束	Running -> Ready	时钟中断保存用户现场并触发重新调度。
主动让出	Running -> Ready	<code>sched_yield</code> 将当前任务重新加入就绪队列。
等待事件	Running -> Blocked	任务登记等待条件后退出可运行集合，由事件、信号或超时唤醒。
任务退出	Running -> Zombie	清理线程私有状态；最后一个线程退出时完成进程级退出。
无可运行任务	进入 idle	先发布当前 CPU 的 idle 位并复查可运行队列，再执行 <code>wfi</code> 或 <code>idle</code> 指令，等待定时器或定向 IPI。

4.1.3 Linux/POSIX 兼容目标

Wll0S 按 Linux 的任务标识关系组织用户执行实体：每个 TCB 拥有唯一 TID，同一 ThreadGroup 的成员共享 TGID；PGID 和 SID 分别描述进程组与会话；parent 和 children 维护父子进程树。调度的基本单位是线程，而进程级退出状态、组停止状态和子进程等待事件由线程组统一维护。

在创建路径上，clone 根据 CLONE_VM、CLONE_FILES、CLONE_FS、CLONE_SIGHAND 和 CLONE_THREAD 等标志决定共享或复制资源，并支持用户栈、TLS、parent/child TID 写回和 clear_child_tid。不共享地址空间时使用写时复制构造子进程；execve 则替换当前进程映像、重建用户栈与寄存器现场，并处理 close-on-exec 文件描述符和信号动作。

在生命周期和调度接口方面，模块实现了 exit、exit_group、wait4、waitid、setpgid、setsid 以及一组 sched_* 接口，同时以 futex、robust list 和线程退出清零机制支撑常见用户线程库。兼容工作的目标不是只返回成功值，而是保证父子关系、资源共享、退出状态、errno、超时和信号中断等行为能够被真实 libc、BusyBox 和测试程序观察。对于 CPU affinity、实时策略和 SCHED_DEADLINE 等能力，文档将明确区分接口兼容与完整调度语义。

4.1.4 模块组成与源码结构

进程与调度功能主要集中在 os/src/task，并由进程系统调用、调度接口、trap 和 timer 代码共同完成。任务模块保存稳定的内核对象和状态机，系统调用层负责把 Linux ABI 参数转换为对象操作，体系结构相关代码负责保存和恢复实际寄存器现场。

源码位置	主要职责
os/src/task/mod.rs	定义 TCB、ThreadGroup、任务状态以及调度、阻塞、唤醒和退出主流程。
os/src/task/task.rs	构造用户任务与内核任务，初始化地址空间、用户栈、TrapFrame 和内核栈。

源码位置	主要职责
<code>os/src/task/manager.rs</code>	维护用户/内核就绪队列、任务注册表，并完成普通任务和实时优先级任务选择。
<code>os/src/task/context.rs</code>	定义 <code>TaskContext</code> ，实现 RISC-V64 与 LoongArch64 的内核任务上下文切换。
<code>os/src/task/wait_queue.rs</code>	统一 I/O、子进程、定时器等等待场景，处理 wait token、超时和唤醒结果。
<code>os/src/platform.rs</code> 与 <code>os/src/cpu.rs</code>	维护 CPU 生命周期、CPU-local 数据、在线与 idle mask、跨核通知以及 TLB shutdown 平台边界。
<code>os/src/syscall/process.rs</code>	实现 <code>clone</code> 、 <code>execve</code> 、 <code>exit</code> 、 <code>exit_group</code> 、 <code>wait4</code> 、 <code>waitid</code> 与 <code>sched_yield</code> 。
<code>os/src/syscall/other.rs</code>	实现进程组、会话、调度参数、CPU affinity、futex 与 robust list 等接口。
<code>os/src/trap</code> 与 <code>os/src/timer.rs</code>	处理用户态进入/返回、时钟抢占、定时等待和到期唤醒。

从控制流看，系统调用层提出“创建、等待或退出”等请求，任务模块修改对象关系和任务状态，调度器选择下一个 TCB，最后由 `trap` 或上下文切换代码恢复执行。后续各节将沿这条控制流展开具体实现。

4.2 执行实体与组织关系

Wll0s 在内核中不直接调度抽象的“进程”，而是调度一个个可以占用 CPU 的 `Task`。用户看到的进程、线程、进程组和会话，则由任务之间的组合关系表达。这样既能让调度器只面对一种对象，也能保留 Linux/POSIX 所需的进程语义。

4.2.1 Task: 内核统一调度实体

```
pub struct TaskControlBlock {
    pub pid: pid::Pid,           // 任务ID
    pub thread_group: Arc<ThreadGroup>, // 所属进程/线程组
}
```

`Task` 是 Wll0s 最基本的执行单元，对应一个 `TaskControlBlock` (TCB)。每个 `Task` 都有独立 TID、运行状态和寄存器现场；就绪、运行、阻塞和退出等状态转换也都以 `Task` 为对象。调度器因此不需要先判断“这是进程还是线程”，只需从就绪队列中选择一个可运行的 TCB。

这里需要区分两个概念：`Task` 回答“谁正在执行”，进程回答“哪些线程共同拥有一组进程级属性”。在 Wll0s 中，后一个问题由 `ThreadGroup` 和若干可共享资源对象共同回答。

4.2.2 用户线程与内核任务

用户线程和内核任务都使用 TCB，但恢复执行的方式不同：

类型	保存的现场	运行方式
用户线程	TrapFrame、用户地址空间和线程私有信号状态	经 trap 返回用户态，系统调用或中断后再次进入内核。
内核任务	TaskContext 和独立内核栈	通过 switch_to 在内核任务与调度器之间切换。

TCB 中的 is_kernel 标记决定任务进入哪类就绪队列以及采用哪条恢复路径。这个差异只涉及执行环境，两者仍共享 TID、任务状态、阻塞唤醒接口和调度入口。

4.2.3 ThreadGroup：进程语义的承载者

```
pub struct ThreadGroup {
    tgid: usize, // Thread Group ID
    members: Mutex<Vec<Weak<TaskControlBlock>>>, // 上锁 + 弱指针引用的列表 记录线程组中的所有 TCB
    exit_code: Mutex<i32>, // 整个进程的退出码
    process_zombie: AtomicUsize, // 表示整个线程组是否已经成为僵尸进程
    stopped: AtomicUsize, // 表示整个线程组是否被停止
    child_wait: Mutex<ChildWaitState>, // 当前进程发生 stop/continue 状态变化后，父进程通过 wait/waitid 可以观察到的事件
}
```

ThreadGroup 把同一进程的用户线程组织起来，保存 TGID、成员弱引用、进程退出码、组停止标记和父进程可观察的状态事件。创建普通子进程时，内核新建 ThreadGroup；使用 CLONE_THREAD 创建线程时，新任务复用父任务的线程组。因此，TCB 描述“一个线程怎样执行”，ThreadGroup 描述“整个进程处于什么生命周期阶段”。

Wll0s 没有为进程重复维护一套完整的 Running/Ready/Blocked 枚举。这三个状态首先属于成员 Task：同一进程可以同时存在正在运行、等待调度和阻塞睡眠的线程。只有必须由父进程统一观察的停止、继续、退出和僵尸状态才在 ThreadGroup 中汇总。理解这一点，可以避免把“一个线程阻塞”误写成“整个进程阻塞”。

状态	进入条件	Wll0s 的处理逻辑
Running	调度器从就绪队列选中某个 Task。	设置 CURRENT_TASK 和 Running，激活用户地址空间并恢复 TrapFrame；内核任务则恢复 TaskContext。时间片结束或 sched_yield 会转入 Ready，等待事件会转入 Blocked。
Ready	任务刚创建、主动让出 CPU，或阻塞事件已经完成。	任务加入用户或内核就绪队列，并通过 TID 集合避免重复入队；被调度器选中后转为 Running。
Blocked	线程等待 I/O、futex、定时器、子进程或信号。	保存现场，记录 block_reason、等待结果和 wait_token，并从可运行集合移除。事件、

状态	进入条件	WIIOS 的处理逻辑
		超时或信号到达且 token 匹配后，线程转回 Ready；同组其他线程仍可继续运行。
Stopped	收到默认动作为停止的 SIGSTOP/SIGTSTP/。SIGTTIN/SIGTTOU	stop_task_group 标记整个 ThreadGroup，删除组内任务的就绪项和等待项，再把所有非僵尸成员设为 Stopped；同时记录停止事件并唤醒等待子进程状态的父任务。
Zombie	最后一个用户线程退出，调用 exit_group，或致命信号终止线程组。	各成员先完成线程级退出；ThreadGroup 只在第一次进程级退出时记录退出码并标记僵尸。内核释放地址空间、文件表等运行期资源，但保留 TGID、退出码和父子关系，等待父进程读取。

从调度角度看，最常见的转换是 Ready -> Running -> Ready 和 Running -> Blocked -> Ready。前者由调度、时间片和主动让出驱动；后者由系统调用登记等待，再由 I/O、futex、定时器或信号唤醒。execve 只替换当前进程的地址空间、用户栈和寄存器入口，并不会创建新的 ThreadGroup，也不会改变进程的 TGID。

停止和继续是线程组级操作。停止信号到达后，stop_task_group 会清除组内成员原有的等待登记，并从就绪队列移除所有实例，防止同一进程仍有线程继续执行；停止信号及其“尚未被父进程读取”的标记保存在 child_wait 中。收到 SIGCONT 时，continue_task_group 清除组停止标记，把所有 Stopped 成员改为 Ready 并重新入队，同时生成 continued 事件。父进程可通过带 WSTOPPED/WCONTINUED 选项的 waitid 观察这些变化。

当前实现采用了较直接的恢复策略：线程组停止时会清除成员原来的阻塞原因，继续时统一恢复为 Ready，而不是逐个恢复到停止前的 Blocked 或 Ready 状态。这能保证作业控制测试稳定推进，但也意味着 WIIOS 尚未保存 Linux 内核中更细粒度的“停止前睡眠状态”。

退出处理分为线程级和进程级两步。exit 先清理当前线程的等待项、clear_child_tid 和 robust futex 状态，再把该 TCB 标记为 Zombie；如果线程组内仍有其他活动线程，进程本身尚未退出。只有全部用户线程都成为 Zombie，或者 exit_group/致命信号主动终止整个线程组时，finish_process_exit 才把 ThreadGroup 标记为进程级僵尸并保存唯一的退出码。

进入进程级僵尸状态后，WIIOS 释放文件锁、用户地址空间和文件描述符等运行期资源，向父进程发送 SIGCHLD，并唤醒阻塞在子进程等待队列中的任务；尚未结束的子进程则移交给 ORPHAN_REAPER。父进程调用 wait4 或 waitid 后取得 TGID 和退出状态：普通等待会从 children 中移除该僵尸进程，完成回收；WNOWAIT 只读取状态而保留父子关系，允许之后再次等待。

因此，从进程生命周期看，可以把主路径概括为：创建后至少有一个 Ready Task，调度后进入活动阶段；所有线程暂时等待时表现为睡眠；组停止信号使整个 ThreadGroup

进入 Stopped；最后一个线程退出后进入进程级 Zombie；父进程成功等待后才真正结束可观察的进程生命周期。

4.2.4 TID、TGID、PGID 与 SID 标识体系

标识	对象	WIIOS 中的含义
TID	Task	由 Pid 分配器为每个任务分配，直接存放在 task.pid 中。
TGID	ThreadGroup	标识一个进程；线程组首个任务通常满足 TID = TGID。
PGID	进程组	多个进程可以拥有同一 PGID，用于作业控制和组信号。
SID	会话	多个进程组可以属于同一会话；会话首进程的 TGID 成为 SID。

因此，gettid 返回当前 Task 的 TID，getpid 返回 ThreadGroup 的 TGID；getpgid 和 getsid 则读取进程组与会话标识。四类编号使用相同的整数形式，但作用层次不同。

4.2.5 父子进程、进程组与会话关系

WIIOS 按“会话包含进程组、进程组包含进程、进程包含线程”的层次组织用户执行实体。parent 与 children 维护父子进程树，PGID 和 SID 描述横向的作业控制关系；setpgid 可以调整进程所属进程组，setsid 则创建新会话并让调用进程同时成为会话首进程和进程组首进程。

父进程退出时，其尚未结束的子进程会移交给 ORPHAN_REAPER。该收养者通常是 init 任务；在评测环境没有独立 init 时，也可以由 harness 承担。这样父子关系不会悬空，后续退出和回收仍有稳定入口。

4.3 核心数据结构与资源组织

WIIOS 将任务数据分成三个层次：TCB 保存一个线程执行所需的状态，ThreadGroup 汇总父进程可观察的进程状态，共享资源对象保存地址空间、文件表和信号动作等进程上下文。三者相互引用，但职责并不重叠。

这种组织方式使调度器只需要操作 TCB，系统调用可以沿 TCB 找到进程资源，退出和等待路径则通过 ThreadGroup 判断整个进程是否结束。共享资源统一使用 Arc<Mutex<...>>：Arc 表示多个任务是否指向同一对象，Mutex 保证对象修改过程的一致性。

4.3.1 任务控制块 TaskControlBlock

TaskControlBlock 是任务模块的总入口。下面只保留能够说明对象关系和执行方式的关键字段：


```
pub struct TaskControlBlock {
    pid: Pid, // TID
    thread_group: Arc<ThreadGroup>, // 所属进程组
    is_kernel: bool, // 是否为内核任务
    task_ctx: KernelCtx, // 内核任务的上下文
    trap_frame: Mutex<Option<TrapFrame>>, // 用户寄存器现场
    memory_set: SharedMemorySet, // 用户地址空间
    fs: SharedFsContext, // 文件系统上下文
    mm: SharedMmContext, // 用户内存布局信息
    signal_actions: SharedSignalActions, // 信号处理动作表
    status: Mutex<TaskStatus>, // 当前调度状态
    wait_token: AtomicUsize, // 等待序号
    affinity_mask: AtomicUsize, // 允许运行的在线 CPU 集合
    blocking_cpu: AtomicUsize, // 同步等待本任务恢复的 CPU
    running_cpu: AtomicUsize, // 当前拥有任务执行现场的 CPU
    inner: Mutex<TaskControlBlockInner>,
}
```

TaskControlBlock 关键字段（省略实现细节）

`pid` 和 `thread_group` 确定线程与进程身份，`task_ctx/trap_frame` 保存内核或用户执行现场，共享资源字段指向进程上下文，`status/wait_token` 服务调度与等待。`affinity_mask` 决定任务可以在哪些在线 CPU 上运行，`running_cpu` 原子声明执行现场所有权，`blocking_cpu` 则记录哪个 CPU 正在同步等待被阻塞的系统调用恢复。`inner` 收纳需要整体加锁修改的父子关系、文件表引用、PGID、SID、资源限制和退出辅助字段。

4.3.2 线程组 ThreadGroup

ThreadGroup 只保存进程级信息：TGID、成员弱引用、统一退出码、进程僵尸标记、组停止标记和 `child_wait` 事件。弱引用让线程组能够遍历成员，又不会因为反向持有 TCB 而延长已退出线程的生命周期。

组内线程可以拥有不同的调度状态；只有停止、继续和整个进程退出等需要由父进程统一观察的结果才在 ThreadGroup 中汇总。具体生命周期处理已在 4.2.3 说明，本节只关注这些数据如何被其他结构引用。

4.3.3 线程私有运行状态

类别	主要字段	作用
执行现场	<code>task_ctx</code> 、 <code>trap_frame</code> 、 <code>is_kernel</code>	区分内核任务与用户线程，并保存上下文切换或 trap 返回所需的寄存器状态。
调度状态	<code>status</code> 、 <code>sched_policy</code> 、 <code>sched_priority</code>	记录 Ready、Running、Blocked、Stopped、Zombie 状态以及普通/实时调度参数。

类别	主要字段	作用
等待状态	<code>block_reason</code> 、 <code>wait_outcome</code> 、 <code>wait_token</code>	描述等待 I/O、子进程、定时器、futex 或信号的原因，并区分唤醒、超时和中断结果。
线程信号状态	<code>signal_state</code>	保存本线程的信号屏蔽字、待处理位图、附加信息和 <code>sigsuspend</code> 旧掩码。
线程退出辅助	<code>clear_child_tid</code> 、 <code>robust list</code>	在线程退出时清零用户 TID 地址、唤醒 futex，并处理可能由退出线程持有的健壮互斥量。

用户线程通过 `trap_frame` 保存用户寄存器，内核任务通过 `task_ctx` 保存返回地址、栈指针和被调用者保存寄存器。两种现场不会同时参与恢复：用户任务的 `trap_frame` 为 `Some`，内核任务主要依赖 `KernelCtx`。把两类任务放进同一种 TCB 后，调度器可以共享状态机和队列接口，只在恢复现场时选择不同路径。

等待状态同样按线程保存。任务准备睡眠时，等待队列先生成新的 `wait_token`，再记录 `block_reason` 并登记 `WaitEntry`。唤醒者必须携带相同 token；如果旧超时在任务已经开始下一次等待后才到达，它会因为 token 不匹配而被丢弃。`wait_outcome` 进一步区分 `Woken`、`TimedOut` 和 `Interrupted`，系统调用据此返回成功、超时或 `EINTR`。

信号动作与信号状态需要分开理解：`signal_actions` 可以由多个线程共享，表示进程安装了什么处理函数；`signal_state` 始终属于当前线程，表示该线程屏蔽了什么、积压了什么信号。新任务继承父线程的屏蔽字，但不会直接复制父线程尚未处理的 `pending` 集合。

`clear_child_tid` 和 `robust list` 是用户线程库与内核之间的退出协议。线程退出时，内核先处理 `robust futex`，再尝试把 `clear_child_tid` 指向的用户地址写为 0 并唤醒等待该地址的 futex，最后清除等待项和寄存器现场并把 TCB 标记为 `Zombie`。这样 `pthread_join` 一类用户态同步可以观察到线程确实结束。

调度策略和静态优先级也放在 TCB 中，而不是 `ThreadGroup` 中。这意味着同一进程的不同线程可以拥有不同的 `SCHED_OTHER/FIFO/RR` 参数；调度器选择的是具体线程，不是整个进程。凭据目前则以每个 TCB 内的 `Mutex<Credentials>` 保存，`clone` 时复制父任务快照，而不是通过共享 `Arc` 自动同步。

4.3.4 进程级共享资源上下文

共享资源没有集中存放在 `ThreadGroup` 内，而是由每个 TCB 持有 `Arc<Mutex<...>>` 引用。两个 TCB 中的字段名称相同，不代表它们一定共享对象：是否共享取决于这些 `Arc` 是否指向同一实例。`clone` 通过标志逐项选择“克隆 `Arc`”或“创建新对象”，因此地址空间、文件表、工作目录和信号动作可以拥有不同的共享边界。

地址空间与内存布局。`memory_set` 保存页表、映射区域和用户栈等真实地址空间内容；`mm` 保存 `program_break`、`mapped_break`、历史低地址 `mmap` 游标和独立高地址 `mmap` 游标。设置 `CLONE_VM` 时，子任务复用父任务的 `memory_set` 与 `mm`，一方建立映射

或写内存会被另一方立即观察到。未设置时，WIIOS 通过 `fork_cow` 创建写时复制地址空间，并复制 `MmContext`；只有实际写入页面时才分离物理页。当前 `CLONE_VFORK` 为避免子任务 `execve` 替换父任务地址空间，也走独立 COW 路径，而不是直接共享父地址空间；与此同时父任务会进入延迟提交的阻塞状态，直到子任务完成 `execve` 或退出。

文件描述符表与文件系统上下文 `fd_table` 是 `Arc<Mutex<FileDescriptorTable>>`，保存打开文件对象、描述符标志和 `close-on-exec` 信息。`CLONE_FILES` 设置时，关闭或新增描述符会影响所有共享者；未设置时会复制描述符表，但表项中的文件对象仍按文件系统对象自身的引用语义存在。`fs` 保存 `cwd`、`root` 和 `umask`，`CLONE_FS` 决定这些路径属性是否联动。把 `fd` 表与 `fs` 上下文拆开，使线程既可以共享打开文件，又可以在需要时拥有独立工作目录。

信号处理动作与线程组。`signal_actions` 保存每个信号的 `handler`、`flags` 和 `mask`；`CLONE_SIGHAND` 设置时复用该表，未设置时复制一份动作。线程自己的 `blocked/pending` 状态仍保存在 `signal_state`。WIIOS 按 Linux 规则检查标志依赖：`CLONE_SIGHAND` 必须与 `CLONE_VM` 一起使用，`CLONE_THREAD` 又要求 `CLONE_SIGHAND`，因此线程组成员必然共享地址空间和信号动作。`CLONE_THREAD` 本身决定是否复用父任务的 `ThreadGroup` 和 `TGID`；未设置时，新任务获得新的 `ThreadGroup`，并作为父进程的子进程登记。

共享与复制结果。下面的“复制”指创建新的外层资源对象，并不等于把所有底层数据立即深拷贝。例如地址空间使用 COW，文件描述符表复制后仍可能引用同一个打开文件对象。

控制标志	共享路径	独立路径
<code>CLONE_VM</code>	复用 <code>memory_set</code> 与 <code>mm</code> ；线程看到同一地址空间。	<code>fork_cow</code> 新建地址空间，复制 <code>MmContext</code> 。
<code>CLONE_FILES</code>	复用 <code>fd_table</code> ；描述符增删和关闭相互可见。	复制 <code>fd</code> 表，后续描述符表修改互不影响。
<code>CLONE_FS</code>	复用 <code>fs</code> ； <code>cwd</code> 、 <code>root</code> 、 <code>umask</code> 联动。	复制 <code>FsContext</code> ，路径属性独立。
<code>CLONE_SIGHAND</code>	复用 <code>signal_actions</code> ；处理动作对共享者一致。	复制动作表；线程 <code>pending</code> 状态始终独立。
<code>CLONE_THREAD</code>	复用 <code>ThreadGroup</code> 与 <code>TGID</code> ，形成同一进程内的新线程。	新建 <code>ThreadGroup</code> 与 <code>TGID</code> ，形成子进程。

资源释放也必须遵守共享关系。进程级退出会遍历线程组成员，但不能对同一个 `Arc` 指向的地址空间或 `fd` 表重复释放。WIIOS 以 `Arc` 指针地址记录已经处理的 `memory_set`、`fd_table` 和 `mm`，每个共享对象只释放一次；随后再清理各 TCB 内的 `clear_child_tid`、`robust list`、定时器和布局快照。`execve` 则先终止同线程组的其他线程，再替换调用线程的地址空间和用户现场，并处理 `close-on-exec` 描述符与信号动作重置。

4.3.5 任务注册表、就绪队列与孤儿进程收养者

任务生命周期不能只靠一张列表维护。Wll0S 分别设置注册表、就绪队列和孤儿收养者，让“系统中有哪些任务”“现在谁可以运行”“父进程退出后谁接管子进程”三个问题由不同结构回答。

结构	持有方式	职责与不变量
<code>TASK_REGISTRY</code>	<code>Vec<Weak<TCB>></code>	记录仍可查询的任务，支持按 TID、TGID、PGID 查找；弱引用失效时惰性清理，不延长任务生命周期。
用户/内核就绪队列	<code>VecDeque<Arc<TCB>></code> 与 TID 集合	只保存可运行任务；队列决定运行次序，集合保证同一 TID 在一条就绪队列中最多出现一次。
<code>EXITED_TASKS</code>	<code>Vec<(TID, TGID)></code>	保留已经观察到的退出身份，辅助区分“从未存在的编号”和“曾存在但已经退出的线程组”。
<code>ORPHAN_REAPER</code>	<code>Option<Arc<TCB>></code>	保存 init 或 harness 收养者；父进程退出时接管尚未结束的子进程。

创建任务时，构造函数先分配 TID 和资源对象，再执行 `register_task` 并加入 `ThreadGroup`；完成身份和组关系登记后，才向用户空间写回 TID 并把新任务放入就绪队列。这个顺序保证任务一旦对调度器可见，其身份、资源和组关系已经完整。

任务阻塞时只从可运行集合中消失：TCB 仍可由等待队列或父子关系持有，`ThreadGroup` 和注册表则保留可升级的弱引用。事件唤醒后，匹配 token 的 TCB 重新进入 Ready 队列。就绪队列使用强引用保证排队任务存活，`BTreeSet` 则防止时钟、I/O、信号和 `futex` 在接近时刻重复加入同一 TID。

线程退出时，内核清除它在就绪队列、核心等待队列、定时器、`futex` 和信号等待队列中的全部实例，将状态置为 `Zombie`，并把 (TID, TGID) 记入 `EXITED_TASKS`。进程级退出还会释放共享资源、发送 `SIGCHLD`，并把子进程移动到 `ORPHAN_REAPER.children`。此时僵尸子进程仍通过父进程的 `children` 保持可观察，父进程才能读取退出码。

父进程通过 `wait4/waitid` 回收后，子进程从 `children` 中移除；当就绪队列、等待队列和其他强引用都消失时，TCB 最终释放。注册表中的 `Weak` 不会阻止释放，下一次 `live_tasks` 扫描会移除无法升级的条目。由此形成“注册用于查询、队列用于运行、父子关系用于等待、收养者用于兜底”的完整资源组织闭环。

4.4 进程与线程生命周期

4.2 与 4.3 已分别说明任务的组织关系和资源结构，本节进一步沿执行顺序说明这些对象如何被创建、替换、退出和回收。Wll0S 把线程作为调度单元，因此线程可以单独创建和退出；父进程可观察的退出、停止和继续事件则由 `ThreadGroup` 汇总为进程级结果。

整个生命周期可以概括为“构造执行现场—登记并进入就绪队列—运行或替换程序映像—转为僵尸状态—由父进程等待回收”。其中 TCB 始终是调度器操作的对象，共享

资源是否复制由 `clone` 标志决定，最终回收则以 `ThreadGroup` 是否完成进程级退出为判断依据。

4.4.1 用户任务与内核任务的创建

Wll0S 为用户任务和内核任务提供不同的构造入口。分别是 `create_user_task(program, argv, env)` 与 `create_kernel_task(entry_function)` 用户任务从 ELF 文件建立地址空间，内核任务从一个内核函数入口建立执行上下文；完成构造后，两者都注册为 TCB，并复用相同的状态机和就绪队列接口。

任务类型	主要构造过程	初始执行现场
用户任务	解析 ELF，映射程序段和用户栈，写入参数及辅助向量，并安装信号 trampoline。	<code>trap_frame = Some(...)</code> ，其中保存用户入口、用户栈指针和参数寄存器； <code>task_ctx</code> 使用零初始化上下文。
内核任务	建立空的用户地址空间对象，分配内核栈，并记录内核函数入口。	<code>trap_frame = None</code> ， <code>task_ctx</code> 的 SP 指向内核栈，RA 指向任务入口函数。

用户任务构造器还会初始化文件表、工作目录、内存布局、信号状态和 root 凭据，并为首个任务创建以自身 TID 为 TGID 的 `ThreadGroup`。内核任务也拥有完整 TCB，但不会构造用户寄存器现场。两类构造器都先分配 TID、注册任务并加入线程组；调用者随后把 Ready 状态的任务放入用户或内核就绪队列，使调度器看到任务时，其身份和资源关系已经完整。

4.4.2 clone/fork 与资源共享机制

Wll0S 使用 `sys_clone` 统一实现线程创建和 `fork` 类进程创建。系统调用首先从当前用户任务复制 `TrapFrame`，再校验共享标志。`CLONE_SIGHAND` 必须与 `CLONE_VM` 同时设置，`CLONE_THREAD` 又要求 `CLONE_SIGHAND`；不满足依赖关系或包含不支持的标志时返回 `EINVAL`。

资源的共享与复制结果已经在 4.3.4 列出。本节关注一次创建操作的实际顺序：

```
let mut child_tf = clone_current_trapframe();
child_tf[RET] = 0; // 子任务看到 clone 返回 0
let memory_set = if share_vm { parent.memory_set.clone() }
                  else { parent.memory_set.lock().fork_cow() };
let thread_group = if clone_thread { parent.thread_group.clone() }
                    else { ThreadGroup::new(child_tid) };
register_task(&child);
thread_group.add_member(&child);
publish_tid_and_enqueue(child);
```

clone 创建主路径（省略错误处理和体系结构差异）

创建子任务时，内核先复制父任务的寄存器现场。调用者如果指定了新栈，就把子任务的栈指针 `SP` 改为该地址；如果设置了 `CLONE_SETTLS`，还会初始化线程本地存储

(TLS) 寄存器。系统调用返回后，父任务得到子任务的 TID，子任务得到 0，因此父子可以判断自己走哪条执行分支。

与 TID 地址相关的标志作用如下：

- `CLONE_PARENT_SETTID`：把新 TID 写入父任务指定的地址。
- `CLONE_CHILD_SETTID`：把新 TID 写入子任务指定的地址。
- `CLONE_CHILD_CLEARTID`：记录一个地址，子任务退出时将其清零，并唤醒等待该地址的线程。

每个任务都有独立的 TID。如果设置 `CLONE_THREAD`，子任务与父任务共享 `ThreadGroup` 和 `TGID`，属于同一进程中的线程，由线程库通过 `join` 回收；否则子任务拥有新的线程组和 `TGID`，作为独立进程加入父任务的 `children`，由 `wait4/waitid` 回收。子任务会继承父任务的调度参数、凭据和信号屏蔽字，但待处理信号、等待状态和退出状态会重新初始化。

对于 `CLONE_VFORK`，WIIOS 没有让父子直接共享地址空间，而是为子任务创建独立的 COW 地址空间，并将子任务插入就绪队列前端优先运行。父任务在子任务发布前先准备一个 `ChildExit` 延迟阻塞，再在子任务完成登记后提交 `Blocked` 状态；子任务执行 `execve` 或退出时通过 `release_vfork_parent` 唤醒父任务。准备、提交和唤醒三步都重新检查等待结果，避免完成事件发生在状态提交之前，也避免 `foreground syscall` 重启时重复创建第二个子任务。这样既保留了 `vfork` 的父进程等待语义，又避免子任务替换父任务正在使用的地址空间。

4.4.3 写时复制与子进程运行现场构造

未设置 `CLONE_VM` 时，`fork_cow` 不会立即复制全部用户内存。它逐个检查映射区域：共享映射继续复用原有页面；私有可写的匿名映射和非共享文件映射复用物理页，并在父子映射上加入软件 COW 标志；只读区域也可以安全共享。只有不适合共享的区域才在创建阶段直接复制。

阶段	页面关系	内核处理
fork 完成后	父子页表指向同一物理页。	私有可写区域带有 COW 标志，实际映射去除写权限；父地址空间重新激活。
读取页面	继续读取共享物理页。	不分配新页面，也不改变引用关系。
第一次写入	写入方需要独立页面。	页错误处理找到对应区域；物理页仍被共享时分配新页并复制一页内容，然后清除本映射的 COW 标志。
只剩一个引用	原物理页已经不再共享。	无需复制页面，直接清除 COW 标志并恢复可写映射。

页错误由 `MemorySet::handle_page_fault` 处理。对于 COW 写异常，它以页为单位分割映射区域，检查页面引用计数，必要时分配并复制 4 KiB 页面，随后重新建立页表映射并激活当前地址空间。因此 `fork` 的初始开销主要是页表和映射元数据，真正发生写入的页面才承担复制成本。

子进程的运行现场与内存复制相互独立。`sys_clone` 先复制父线程当前 `TrapFrame`，将返回值寄存器设为 0，并完成系统调用后的 PC 推进；父线程仍由系统调用返回路径获得子 TID。可选的用户栈和 TLS 修改也只作用于子现场。子任务进入 Ready 队列后，调度器直接恢复这份 `TrapFrame`，使它看起来像从父任务调用 `clone/fork` 的位置继续执行。

4.4.4 `execve` 进程映像替换

`execve` 不创建新进程，而是把当前进程正在运行的程序完全替换成另一个程序。为避免装载失败破坏原进程，WIIOS 把执行过程分成准备和提交两个阶段。

阶段	处理内容
准备新映像	先从旧地址空间读取 <code>path</code> 、 <code>argv</code> 和 <code>envp</code> ，按 <code>cwd/root</code> 解析路径；处理脚本解释器和 ELF <code>PT_INTERP</code> ，装载目标程序及动态解释器，建立用户栈，并写入 <code>argc</code> 、 <code>argv</code> 、 <code>envp</code> 和 <code>auxv</code> 。此时失败可以直接返回错误，旧映像仍然有效。
提交替换	新地址空间完全构造成功后，终止同线程组的其他线程，解除旧共享内存关系，重置信号处理动作和 <code>robust list</code> ，关闭 <code>close-on-exec</code> 描述符，重置堆与 <code>mmap</code> 布局，最后替换 <code>memory_set</code> 和 <code>TrapFrame</code> 。

新的 `TrapFrame` 保存 ELF 入口、用户栈指针和 `argc` 等入口参数。替换后内核立即激活新页表，并通知 `trap` 返回路径不要再次推进系统调用 PC，否则会跳过新程序的第一条指令。地址空间、用户栈和用户寄存器全部改变，但当前任务的 TID/TGID、父子关系、PGID、SID 以及未设置 `close-on-exec` 的文件描述符保持不变。

对于多线程进程，当前实现会先调用 `terminate_thread_group_peers_for_exec`，把同一 `ThreadGroup` 中除调用者外的线程执行退出清理，再由调用线程承载新映像。这样旧程序的其他线程不会继续使用已经被替换的地址空间和信号动作。

4.4.5 线程退出与 `exit_group`

`exit` 与 `exit_group` 最终都进入任务模块的退出路径，但作用范围不同。普通 `exit` 只结束调用线程；当线程组中所有用户线程都成为 `Zombie` 后，才继续完成进程级退出。`exit_group` 则遍历 `ThreadGroup` 成员，对所有用户线程执行退出清理，并立即提交一次进程级退出结果。

入口	作用范围	进入进程级退出的条件
<code>exit</code>	只清理当前 TCB。	<code>all_user_members_zombie()</code> 为真，即调用线程是线程组中最后一个退出的用户线程。
<code>exit_group</code>	清理 <code>ThreadGroup</code> 中全部用户 TCB。	所有成员处理完毕后直接调用 <code>finish_process_exit</code> ；进程僵尸标记保证该过程只提交一次。
内核任务退出	结束当前内核 TCB，并切回调度器。	内核任务不形成父进程通过 <code>wait</code> 观察的用户进程级 <code>Zombie</code> 。

单个线程的 `finish_task_exit` 按固定顺序工作：首先遍历 `robust list`，标记退出线程可能遗留的 `futex`；随后把 `clear_child_tid` 指向的用户地址写为 0，并唤醒等待该地址的 `futex`。接着移除任务在通用等待队列、定时器、`futex` 和信号等待结构中的登记，保存退出码，清空 `TrapFrame`、阻塞原因与唤醒结果，最后把 TCB 状态设为 `Zombie` 并记录退出的 TID/TGID。

进程级退出还需要释放线程组共享资源。Wll0S 恢复内核页表后，按 `Arc` 指针去重地址空间、文件描述符表和内存布局对象，确保同一共享对象只被释放一次；同时释放文件锁、共享内存挂接和区间定时器。线程级 `Zombie` 表示该调度实体不能再次运行，进程级 `Zombie` 则表示父进程已经可以观察统一退出码。

4.4.6 僵尸进程、`wait4/waitid` 与资源回收

最后一个线程退出或 `exit_group` 完成后，`finish_process_exit` 在 `ThreadGroup` 中设置进程僵尸标记和统一退出码，并向父进程发送 `SIGCHLD`。此时可执行资源已经释放，但子进程对象仍保留在父任务的 `children` 中，使父进程能够查询它的 TGID 和退出状态。

接口	目标与返回内容	当前支持的等待语义
<code>wait4</code>	等待任意子进程或指定 TGID；成功返回子进程 TGID，并写入编码后的退出状态。	支持 <code>pid=-1</code> 、 <code>pid=0</code> 、正 TGID 和 <code>WNOHANG</code> 。当前 <code>rusage</code> 参数尚未填充。
<code>waitid</code>	按 <code>P_ALL</code> 、 <code>P_PID</code> 或 <code>P_PGID</code> 选择目标，并向用户空间写入 <code>siginfo</code> 。	支持 <code>WEXITED</code> 、 <code>WSTOPPED</code> 、 <code>WCONTINUED</code> 、 <code>WNOHANG</code> 和 <code>WNOWAIT</code> ； <code>WNOWAIT</code> 只观察事件而不立即消费。

等待函数首先扫描调用进程各线程保存的 `children`。找到匹配的进程级 `Zombie` 时，`reap_zombie_child` 从列表中移除它并返回退出结果；`waitid(WNOWAIT)` 则使用只读查询保留该对象。如果存在匹配子进程但尚无所需事件，`WNOHANG` 立即返回 0，否则当前线程以 `ChildExit` 原因进入等待队列，并在子进程退出、停止或继续时被唤醒后重新检查条件。没有匹配子进程时返回 `ECHILD`；收到 `EINTR` 型唤醒时，等待路径也会先重新检查子进程状态，避免在事件竞争窗口中漏掉已经产生的退出结果。

“移出 `children`”是逻辑回收点。之后，随着就绪队列、等待队列以及其他强引用消失，子进程 TCB 才最终释放；任务注册表保存的是 `Weak`，不会延长其生命周期，并会在后续扫描时清理失效条目。因此 Wll0S 把“停止执行”“释放运行资源”“父进程取得退出状态”和“对象最终析构”分成了四个明确阶段。

4.4.7 孤儿进程重托管机制

父进程退出时，它可能仍持有正在运行或已经成为 `Zombie` 的子进程。`finish_process_exit` 会遍历线程组全部成员，取出各自 `children` 中的任务，统一交给 `ORPHAN_REAPER`。正常启动环境通常由 `init` 承担收养者，评测环境也可以把 `harness` 注册为 `reaper`。

重托管时，内核先把子进程的 `parent` 更新为 `reaper`，再把同一个子进程对象加入 `reaper.children`。运行中的孤儿以后退出时会向新的父进程报告；已经成为 `Zombie` 的

孤儿也仍留在新的 `children` 中，可由 `reaper` 的 `wait4/waitid` 回收。转移完成后会广播唤醒子进程等待者，避免 `reaper` 已经睡眠却错过现有僵尸事件。

若系统尚未设置收养者，Wll0S 会把子进程的 `parent` 置空，避免保留指向已退出父任务的引用；正常配置下则由 `ORPHAN_REAPER` 提供最终回收入口。这样父子关系不会悬空，也不会因父进程提前结束而遗失子进程。

4.5 调度器总体设计

Wll0S 的调度器以 `TaskControlBlock` 为基本对象，负责把就绪队列中的任务转换为 CPU 上的实际执行流。它不仅需要决定“下一个运行谁”，还必须同时维护任务状态、队列成员关系、当前任务指针、地址空间和寄存器现场的一致性。调度器与 `trap`、`timer`、信号和等待队列共同构成闭环，使任务能够在运行、让出、阻塞、唤醒和退出之间转换。

Wll0S 当前面向 SMP 环境，采用两类全局就绪队列和 CPU-local 调度状态。每个在线 CPU 都可以选择满足自身 `affinity` 的任务，调度器优先选择有效实时优先级最高的候选；普通任务和同优先级任务按队列顺序轮流运行。每个任务默认运行 10 ms，时间片结束或主动让出 CPU 后重新排队；阻塞任务则等事件唤醒后再恢复。用户任务以 `TrapFrame` 保存用户态现场，内核任务以 `TaskContext` 保存内核执行现场；两类任务经过同一个选择入口，但在真正恢复执行时进入不同路径。下面依次说明调度主流程、队列组织、普通与实时任务选择、跨核唤醒以及评测前台驱动机制。

4.5.1 调度器组成与运行流程

`run_tasks` 是普通运行模式下的调度入口，它调用 `run_next_task` 取得当前 CPU 可以调度的任务。选择成功后，任务先通过 `try_start_running` 原子取得执行所有权并从 `Ready` 转为 `Running`，调度器再把它保存到当前 CPU 的 `CURRENT_TASK`。用户任务需要激活所属地址空间、处理待决信号并通过 `run_user_task` 返回用户态；内核任务则通过 `switch_to` 从当前 CPU 的调度器槽切换到任务自己的 `TaskContext`。

用户任务运行到下一次系统调用、中断或异常后重新进入内核。调度器先恢复内核页表并保存更新后的 `TrapFrame`，再检查任务状态：仍为 `Running` 或 `Ready` 的任务变为 `Ready` 并重新入队；已经进入 `Blocked`、`Stopped` 或 `Zombie` 的任务不会回到就绪队列。内核任务主动让出或阻塞时，则通过保存的调度器上下文切回 `run_next_task` 所在执行流。

```
fn run_next_task() {
    let Some(task) = fetch_dispatchable_task() else {
        // 当前 CPU 无可运行任务，进入空闲等待
        return idle_loop();
    };

    // 状态和 CPU 所有权必须同时满足，防止同一现场被并发恢复
    if !task.try_start_running() {
        return;
    }
    *CURRENT_TASK.lock() = Some(task.clone());
}
```



```

if task.is_kernel {
    switch_to_kernel_task(&task);
} else {
    task.activate_memory_set_if_needed();
    handle_pending_signal(&task); // 处理用户任务尚未响应的信号
    run_user_task(&mut task.trap_frame); // 进入用户态，发生系统调用、中断或异常时
    返回
    restore_kernel_page_table();
    task.release_running_cpu();
    requeue_after_user_run(task); // 未阻塞或退出的任务重新加入就绪队列
}
}

```

代码 4-1 调度主路径的结构化示意

上述流程把“选择任务”和“恢复任务”分开：`manager` 只负责队列与优先级，`run_next_task` 负责状态转换和运行路径，`context` 与 `trap` 层负责体系结构寄存器。这样的分工使队列策略可以独立调整，同时保证 RISC-V64 与 LoongArch64 共享同一套调度控制逻辑。

4.5.2 用户任务与内核任务双就绪队列

任务管理器分别维护 `USER_READY_QUEUE` 和 `KERNEL_READY_QUEUE`。两者都保存 `Arc<TaskControlBlock>`，但服务的执行对象不同：用户队列保存带用户地址空间和 `TrapFrame` 的任务，内核队列保存拥有独立内核栈和 `TaskContext` 的内核任务。`add_task` 根据 `is_kernel` 自动选择队列，从而避免调用者直接操作具体队列。

队列	任务特征	恢复方式
<code>USER_READY_QUEUE</code>	用户进程主线程或由 <code>clone</code> 创建的用户线程，拥有 TID、地址空间和用户寄存器现场。	激活 <code>MemorySet</code> ，恢复 <code>TrapFrame</code> ，通过 <code>trap</code> 返回用户态。
<code>KERNEL_READY_QUEUE</code>	<code>harness</code> 、内核工作任务等仅在内核态运行的执行流，拥有独立内核栈。	通过 <code>TaskContext</code> 和汇编 <code>switch_to</code> 恢复内核执行现场。

分离队列的主要原因并不是把用户任务和内核任务设计成不同的调度实体，而是隔离两套恢复机制。两类对象仍然共享 TCB 状态机、TID 分配、入队接口和调度入口。当前两条队列是多 CPU 共同访问的全局队列，而不是 per-CPU run queue；队列项不保存固定 CPU 归属，`fetch_from_queue` 在每次出队时把任务 `affinity` 与 `online mask` 相交，并跳过当前 CPU 不允许运行的任务。

4.5.3 就绪队列去重与任务注册机制

每个就绪队列由 `VecDeque` 和 `BTreeSet` 共同组成。`VecDeque` 保存可运行次序，`BTreeSet` 保存已经入队的 TID。任务进入队列时只有在 TID 尚未出现的情况下才会真

正插入；任务被取出或删除时，两种结构同步移除对应 TID。这样可以避免时钟、I/O、信号和 `futex` 在相近时刻重复唤醒同一个任务，破坏“一项任务最多存在一个就绪队列项”的不变量。

```
struct ReadyQueue {
    tasks: VecDeque<Arc<TaskControlBlock>>,
    queued_pids: BTreeSet<usize>,
}
fn push_back(&mut self, task: Arc<TaskControlBlock>) {
    if self.queued_pids.insert(task.pid.0) {
        self.tasks.push_back(task);
    }
}
fn remove_at(&mut self, index: usize) -> Option<Arc<TaskControlBlock>> {
    let task = self.tasks.remove(index)?;
    self.queued_pids.remove(&task.pid.0);
    Some(task)
}
```

代码 4-2 就绪队列的成员去重

调度器取任务时还会检查实时状态。若队列中残留 `Zombie`、`Blocked` 或 `Stopped` 项，`fetch_from_queue` 会将其删除而不是运行。该检查是队列不变量的第二层保护：正常路径应在状态变化时同步维护队列，而选择路径仍对过期队列项进行防御性清理。

除就绪队列外，`TASK_REGISTRY` 使用 `Weak<TaskControlBlock>` 保存任务注册信息。弱引用允许系统按 TID、TGID 或 PGID 查询存活任务，同时不会因为注册表本身延长已经退出任务的生命周期；查询时无法升级的弱引用会被清除。就绪队列回答“谁可以运行”，注册表回答“系统中有哪些任务”，两者职责不能混用。

4.5.4 可运行任务发布与跨核唤醒

多核环境中的入队不仅要维护队列，还要决定是否唤醒其他 CPU。Wll0s 区分“当前 CPU 接下来就会继续调度”的本地重排队和“另一个执行流新产生了可运行任务”的外部发布。时间片结束、系统调用返回和内核任务切回等本地路径使用 `add_task_local`，只要当前 CPU 满足 `affinity`，就不发送远程 IPI；新任务、外部事件唤醒和其他 CPU 产生的 `runnable` 则使用带通知的发布路径。

外部发布先就在绪队列锁内完成去重与插入，释放队列锁后，再从 `online`、`idle` 与 `affinity` 三个 `mask` 的交集中原子选择一个 CPU 并发送 IPI。它不会向全部 `idle` CPU 广播，因为一个新队列项只需要一个执行者。若当前 CPU 不在任务 `affinity` 中，即使调用的是本地入队接口，也会自动退回到外部通知路径。

发布场景	通知方式	保证的不变量
本地重排队	当前 CPU 可运行时不发送 IPI	避免时间片和 <code>syscall</code> 边界反复唤醒其他空闲 CPU。
外部 <code>runnable</code>	只通知一个满足 <code>affinity</code> 的 <code>idle</code> CPU	队列项先可见，IPI 后发送，且不在队列锁内执行平台通知。

发布场景	通知方式	保证的不变量
当前 CPU 不满足 affinity	转为外部定向通知	任务不会停留在当前 CPU 无法消费的队列中。
线程组或 affinity 批量变化	按需要广播重新评估	多个任务可能同时获得新的可运行条件。

idle CPU 在真正执行 `wfi` 或 `idle 0` 之前先发布自己的 idle 位，再复查满足 affinity 的就绪队列。若任务更早入队，第二次检查会直接取得它；若任务稍后入队，生产者会看到 idle 位并发送 IPI。这个顺序关闭了“队列检查已经结束、CPU 尚未睡下”之间的丢失唤醒窗口。

4.5.5 普通任务的轮转调度

对于 `SCHED_OTHER`、`SCHED_BATCH`、`SCHED_IDLE` 以及当前只提供接口兼容的 `SCHED_DEADLINE`，`effective_sched_priority` 返回 0。调度器因此选择队列中最靠前的可运行任务。任务完成一次运行片段后，`requeue_after_user_run` 将仍可运行的任务放到队尾；下一次选择会从新的队首开始，形成基于 FIFO 队列的简化轮转调度。

普通模式的定时器时间片为 10 ms。时钟中断首先唤醒已经到期的定时等待任务，再保存当前用户现场并调用 `suspend_current_and_run_next`。任务也可以通过 `sched_yield` 主动放弃 CPU。无论由时间片还是主动让出触发，只要任务仍可运行，都会经历 `Running -> Ready` 并重新入队。

触发条件	状态处理	队列行为
时间片到期	<code>Running -> Ready</code>	保存 <code>TrapFrame</code> ，放回队尾，再选择下一任务。
<code>sched_yield</code>	<code>Running -> Ready</code>	当前任务主动让出；普通模式下立即进入下一次调度。
系统调用或异常返回	按处理结果决定	仍可运行则回到队尾；阻塞、停止或退出则不入队。
等待事件	<code>Running -> Blocked</code>	从可运行集合移除，直到等待队列、定时器或信号完成唤醒。
任务退出	<code>Running -> Zombie</code>	清除全部就绪队列实例，不再参与选择。

这种实现能够让多个普通 CPU-bound 任务交替前进，但它没有 nice 权重、虚拟运行时间或动态时间片等机制，因此更接近基础 round-robin，而不是 Linux CFS。文档使用“轮转调度”描述当前行为，不把 `SCHED_OTHER` 接口等同于完整 Linux 普通调度类。

4.5.6 FIFO/RR 实时优先级选择

当任务策略为 `SCHED_FIFO` 或 `SCHED_RR` 时，TCB 中 1 到 99 的 `sched_priority` 成为有效优先级。`fetch_from_queue` 线性扫描就绪队列，选择有效优先级最高的任务；同一优先级下保留原有队列先后关系。调度参数在选取时重新读取，因此任务已经入队后通过 `sched_setscheduler` 或 `sched_setparam` 修改优先级，下一次选择仍能观察到新值。

当前实现还维护 `RT_RUNS_SINCE_NORMAL` 计数。最高优先级实时任务被连续选择达到预算后，如果队列中存在次高实时优先级任务，调度器会安排一次次高优先级任务。这一机制用于限制最高优先级任务完全垄断其他实时任务，但它属于 Wll0S 当前的公平性折中，并不符合 Linux `SCHED_FIFO` 对严格优先级的完整定义。

策略	当前有效优先级	当前实现语义
<code>SCHED_FIFO</code>	1-99	参与实时优先级选择，但会受到次高优先级运行预算影响。
<code>SCHED_RR</code>	1-99	与 FIFO 共用选择逻辑；接口返回 10 ms RR 间隔，但尚无独立的 per-task quantum 统计。
<code>SCHED_OTHER/BATCH/IDLE</code>	0	按普通就绪队列次序轮转，尚未区分各普通策略的权重和时间片。
<code>SCHED_DEADLINE</code>	0	保留参数与查询接口，没有 deadline、runtime、period 驱动的选择器。

因此，Wll0S 已经能够保存调度策略、校验实时优先级并影响任务选择，但 FIFO 与 RR 尚未形成严格不同的运行规则，也没有按优先级建立多级队列。当前线性扫描实现适合任务规模较小、共享队列长度受控的 SMP 环境；若后续扩大并发规模，可将实时任务改为按优先级组织的队列，并分别实现 FIFO 抢占条件和 RR 时间片消耗。

4.5.7 用户任务与内核任务的调度次序

双队列之间采用明确的用户任务优先顺序。`fetch_dispatchable_task` 首先尝试从用户队列取得任务，只有用户队列没有可调度任务且当前不处于 foreground driver 时，才从内核队列选择任务。这个顺序让用户程序、shell 和评测用例能够持续推进，也简化了用户地址空间运行路径。

运行场景	选择顺序	说明
普通调度模式	用户队列 -> 内核队列	用户任务为空时，内核任务才获得选择机会。
前台评测模式	仅前台用户队列	调度入口没有用户任务时直接返回 harness，不继续抽取内核任务。
用户任务仍在活跃调用链中	跳过该 TID 后继续查找	防止前台阻塞路径递归选择同一活跃任务；被跳过项随后重新入队。

这种次序是当前实现策略，而不是由调度优先级统一计算出的结果。即使内核任务设置了实时优先级，只要用户队列持续非空，它也可能较晚运行。因此，用户/内核队列优先关系和队列内部的 FIFO/RR 优先级是两个不同层次：前者决定从哪一类队列取任务，后者决定在选定队列内部取哪一个任务。

该设计存在饥饿风险，后续若引入必须周期运行的内核工作线程，应增加跨队列配额、统一优先级域或专门的内核任务保障机制，避免用户队列长期非空时内核维护任务得不到运行。

4.5.8 空闲处理与前台测试驱动路径

普通运行模式下，当两个就绪队列都没有当前 CPU 可以运行的任务时，`run_next_task` 进入 `idle_loop`。CPU 先通过平台层发布 idle 位并再次检查队列，随后 RISC-V64 执行 `wfi`，LoongArch64 执行 `idle 0`。定时器或 runnable 发布 IPI 到达后，内核清除 idle 位、处理到期定时器并重新检查就绪队列。如果等待事件产生了新的 Ready 任务，调度器再次进入任务选择流程。

评测使用的 foreground driver 则采用同步驱动方式。harness 将测试根任务加入用户队列，在循环中选取一个用户任务，激活其地址空间并运行到下一次 trap，再保存现场和重新入队。前台模式维护独立 deadline，并使用 50 ms 的定时器间隔；时钟 trap 负责唤醒到期等待者和处理信号，但不在 trap 内直接调用普通抢占路径，外层 harness 循环负责继续推进任务。

比较项	普通调度模式	前台测试驱动
驱动入口	<code>run_tasks -> run_next_task</code>	harness 的 <code>run_user_task_foreground</code> 循环。
定时器间隔	10 ms	50 ms，降低静态 libc 启动阶段频繁返回 harness 的开销。
时钟到期	保存当前现场并触发重新调度。	唤醒超时任务并返回外层驱动，不在 trap 中切走当前测试树。
无可运行任务	执行 <code>wfi</code> 或 <code>idle 0</code> 等待中断。	返回或继续检查 deadline，由 harness 决定结束、超时或下一次运行。
内核任务	用户队列为空后可从内核队列选择。	前台用户任务循环不抽取内核队列。
结束条件	调度循环通常不返回。	根任务及其用户任务结束，或 deadline 到期后清理任务树。

前台驱动复用真实的 TCB、地址空间、系统调用、信号和退出语义，因此测试程序观察到的仍是内核实现；但它对任务选择、时钟抢占和空闲处理施加了额外控制。文档将其作为评测基础设施单独说明，避免把 foreground 行为误认为通用调度器的默认语义。

4.6 阻塞、唤醒与等待机制

任务等待数据、时间或其他进程时，如果一直循环检查条件，就会白白占用 CPU。Wll0S 的处理方式是当前任务从运行状态改为 `Blocked`，记录它正在等待什么以及哪个 CPU 正在同步等待本次系统调用恢复，然后让调度器运行其他任务。等事件真正发生后，内核再把它改回 `Ready`；如果原 owner CPU 仍在等待，就向该 CPU 发送定向 IPI，否则把任务重新发布到就绪队列。

这套机制需要解决两个问题：一是任务不能错过发生在“检查条件”和“开始睡眠”之间的事件；二是超时、信号和正常事件可能同时到达，但一次等待只能得到一个有效结果。Wll0S 通过等待队列、条件重检和 `wait_token` 共同解决这些问题。

4.6.1 等待原因与等待结果

TCB 使用 `block_reason` 记录任务为什么被阻塞，使用 `wait_outcome` 记录它为什么被唤醒。原因用于选择和清理等待队列，结果则决定系统调用恢复后应该返回成功、超时还是中断。

阻塞原因	常见使用场景	恢复条件
Io	<code>pipe</code> 、 <code>eventfd</code> 、 <code>poll/epoll</code> 和文件锁。	数据可读、缓冲区可写、fd 状态变化或文件锁被释放。
ChildExit	<code>wait4/waitid</code> 等待子进程事件。	目标子进程退出、停止或继续，父任务重新检查 <code>children</code> 。
Timer	<code>nanosleep</code> 以及只等待超时的调用。	时钟达到登记的 <code>deadline</code> 。
Futex	线程同步和 <code>pthread_join</code> 。	指定 <code>futex</code> 地址被唤醒、等待超时或收到可处理信号。
Signal	<code>sigtimedwait</code> 和 <code>sigsuspend</code> 。	等待集合中出现信号，或出现当前线程可以处理的信号。

一次等待最终得到三种结果之一：

- **Woken**：目标事件已经发生，可以重新检查资源或继续执行。
- **TimedOut**：等待超过了截止时间。
- **Interrupted**：可处理信号到达，本次阻塞被中断。

通用 `WaitQueue` 的每个等待项保存 TCB、`wait_token` 和可选的 `WaitKey`。`WaitKey` 由“对象编号”和“事件编号”组成，例如某个 `pipe` 的“可读”事件。带 `key` 的唤醒只处理真正等待该对象的任务；`poll` 一类需要重新检查多个 fd 的任务可以使用无 `key` 等待。

4.6.2 一次阻塞的完整过程

以“从空 `pipe` 读取数据”为例，一次阻塞大致经过下面几个步骤：

- 系统调用先检查 `pipe`。如果已经有数据，直接读取，不进入等待队列。
- 如果暂时无数据，内核为当前任务生成新的 `wait_token`，清除上一次等待结果，并记录 `BlockReason::Io`。
- 内核把任务登记到 I/O 等待队列，然后再次检查 `pipe` 是否仍然为空。
- 条件仍不满足时，保存用户 `TrapFrame`，把当前 CPU 写入 `blocking_cpu`，将任务状态改为 `Blocked`，清除本 CPU 的 `CURRENT_TASK`，并让调度器选择其他任务。
- 写端写入数据后，按 `pipe` 的 `WaitKey` 找到读等待者。有效等待者被改为 `Ready`；存在同步 `owner` 时向对应 CPU 发送 IPI，否则重新加入用户就绪队列。
- 任务再次运行后删除自己的等待项和超时项，根据 `wait_outcome` 判断本次等待是正常唤醒、超时还是信号中断，然后重新执行读取检查。

这里最重要的是“先登记，再重检”。假设读取线程第一次检查时 `pipe` 为空，而写线程恰好在它加入等待队列之前写入数据；如果读取线程直接睡眠，就可能永远等不到下一次通知。Wll0S 在登记后调用 `should_sleep` 再检查一次条件：数据已经到达就撤销等待，不再进入 `Blocked`，从而关闭这个丢失唤醒的窗口。

信号也会在真正阻塞前再检查一次。如果已经存在未屏蔽信号，内核会移除刚登记的等待项并返回 `EINTR`，避免任务带着一个可以立即处理的信号继续睡眠。

4.6.3 wait_token 与并发事件处理

`wait_token` 可以理解为“一次等待的流水号”。任务每开始一次新的等待，编号就加一；等待队列和定时器都保存当时的编号。唤醒时，内核只有在编号仍与 TCB 中的当前值相同时才接受这个事件。

例如，某任务的第一次定时等待使用 token 11。它被正常事件提前唤醒后，很快开始第二次等待，token 变成 12。此时第一次等待的超时事件才到达，但它携带的仍是 11。内核发现编号不匹配后会丢弃这个旧事件，因此第二次等待不会被错误唤醒。

token 还处理“事件先于睡眠”的情况。任务已经登记等待项并设置 `block_reason`，但尚未来得及把状态改为 `Blocked` 时，事件可能已经发生。`wake_task_token_with` 会先核对 token；如果任务还没有正式睡眠，就把结果写入 `wait_outcome`。随后阻塞入口看到结果已经存在，会跳过睡眠。这样事件无论发生在阻塞前还是阻塞后，都只会生效一次。

任务正式处于 `Blocked` 时，有效唤醒先把状态改为 `Ready`、清除 `block_reason` 并写入 `wait_outcome`。如果 `blocking_cpu` 仍指向某个 owner，唤醒者只向该 CPU 发送 IPI，让原系统调用在同一同步调用链中恢复；owner 已经释放时，任务才重新加入全局就绪队列。就绪队列自身还会按 TID 去重，因此多个接近同时到达的唤醒不会产生重复队列项。

4.6.4 blocked-owner 事件等待

部分系统调用在内核调用链中同步等待事件，当前 CPU 仍是该任务 trap frame 和内核执行边界的 owner。旧实现如果没有其他可运行任务，会不断扫描就绪队列并执行短暂空转；这种方式虽然能够观察唤醒，却在多核和 QEMU TCG 环境中产生大量无效锁访问。当前实现改为事件驱动的 blocked-owner 等待。

owner CPU 先发布 idle 位，再第二次检查被阻塞任务状态和用户就绪队列。若另一个任务已经 Ready，就由该 CPU 直接运行一个用户边界；若仍无工作，则进入 `wfi` 或 `idle 0`。新的 runnable 在稍后发布时会通过 idle mask 触发 IPI，被阻塞任务自身完成时则通过 `blocking_cpu` 向 owner 发送定向 IPI。owner 醒来后继续调用 `wake_expired_timers` 并重新检查状态，因此定时等待不会因为 CPU 休眠而丢失。

这条路径保持三个所有权不变量：`CURRENT_TASK` 只描述本 CPU 当前执行对象，`running_cpu` 保证用户现场不会并发运行，`blocking_cpu` 保证同步等待完成后由正确的 CPU 恢复。它优化的是空转和唤醒协议，不改变 `WaitQueue`、`timer`、`futex` 或系统调用返回语义。

4.6.5 I/O、定时器、futex 与信号等待

不同系统调用等待的对象不同，但都遵循“登记条件—进入 Blocked—事件唤醒—恢复后重检”的主流程。

等待类型	登记方式	唤醒方式
I/O	pipe、eventfd 和文件锁使用 <code>WaitKey</code> ；poll/epoll 可以登记带 deadline 的通用 I/O 等待。	读写状态或锁状态改变后按 key 唤醒；poll/epoll 恢复后重新扫描全部目标 fd。
定时器	<code>SLEEP_QUEUE</code> 保存睡眠任务， <code>TIMER_WAITERS</code> 另外保存 deadline、TCB 和 token。	时钟中断调用 <code>wake_expired_timers</code> ，把已经到期的任务以 <code>TimedOut</code> 结果唤醒。
futex	按用户地址、私有/共享 key 和 bitset 登记到 <code>FUTEX_WAITERS</code> ；登记前后各检查一次用户值。	<code>FUTEX_WAKE</code> 可唤醒指定数量的匹配任务， <code>requeue</code> 可以把等待者移到另一个地址；超时和信号也能结束等待。
信号	<code>sigtimedwait/sigsuspend</code> 使用独立的 <code>SIGNAL_WAIT_QUEUE</code> ，并按照线程信号屏蔽字判断条件。	信号入队后唤醒信号等待者；如果某个阻塞任务存在可投递信号，还会以 <code>Interrupted</code> 结果定向唤醒它。
子进程	<code>wait4/waitid</code> 在 <code>CHILD_WAIT_QUEUE</code> 上等待，但退出状态仍保存在 <code>ThreadGroup</code> 和父任务的 <code>children</code> 中。	子进程退出、停止或继续时广播唤醒；父任务恢复后重新查找目标事件，不能只依赖一次通知。

futex 比普通等待多一步数值校验。内核只有在用户地址中的值仍等于调用者给出的期望值时才睡眠，否则立即返回 `EAGAIN`。任务加入 `FUTEX_WAITERS` 后还要再次读取该值，防止另一个线程刚刚修改了 futex，却没有看到尚未完成登记的等待者。

定时等待不会只依赖固定的 10 ms 调度时间片。加入超时项时，内核会根据最近 deadline 重新设置下一次时钟触发时间；每次时钟处理、空闲循环或前台调度循环都会检查到期项。因此即使就绪队列中一直存在其他任务，超时等待也能够被及时观察。

4.6.6 唤醒返回与退出清理

同一种 `WaitOutcome` 在不同系统调用中会转换成不同的用户可见结果。正常事件通常让系统调用重新检查条件；超时时，poll 返回 0，futex 返回 `ETIMEDOUT`，`sigtimedwait` 返回 `EAGAIN`；信号中断通常表现为 `EINTR`。`wait4/waitid` 收到中断型唤醒后会先重新检查子进程状态，避免退出事件和信号同时发生时漏掉已经产生的结果。

任务恢复后会主动删除本次等待项；带 deadline 的等待还会删除对应定时器。删除操作同时使用 TID 和 token，因此不会误删同一任务之后建立的新等待。

任务退出、被线程组终止或进入组停止状态时，不能把等待项留在全局队列中。Wll0s 的 `purge_wait_state_for_task` 会统一清理：

- 用户/内核就绪队列中的残留实例；
- I/O 和子进程通用等待项；

- 定时等待和 `timeout`;
- `futex` 等待项;
- 信号等待项。

等待项持有 TCB 的强引用。如果退出时不清理，即使任务已经成为 `Zombie`，等待队列仍可能让对象长期存活，旧事件也可能再次尝试唤醒它。统一清理后，`Zombie` 不会重新进入 `Ready` 队列，TCB 也能在父进程完成回收、其他强引用消失后正常释放。

从状态转换看，等待队列的正常出口是 `Ready`，不是 `Zombie`。只有任务执行 `exit`、被 `exit_group` 终止或收到致命信号时，退出清理才会把它标记为 `Zombie`；作业控制停止信号则把线程组成员改为 `Stopped`，收到 `SIGCONT` 后再回到 `Ready`。

4.7 上下文切换与抢占机制

上下文切换解决的是“一个任务暂停后，下一次怎样从原位置继续运行”。Wll0S 把线程作为调度基本单位，因此每个 Task 都需要保存自己的运行现场；SMP 环境还要求明确哪个 CPU 当前拥有这份现场。调度器切换任务时，先保存当前任务的现场和状态、释放旧 CPU 所有权，再恢复下一个任务的现场；对用户任务还要同时切换地址空间。

用户任务和内核任务的运行方式不同，Wll0S 没有强行使用同一种上下文结构。用户任务通过 `TrapFrame` 在用户态和内核态之间往返，内核任务则通过更精简的 `TaskContext` 在任务与调度器之间切换。

4.7.1 两类运行现场：TrapFrame 与 TaskContext

`TrapFrame` 保存用户程序被系统调用、中断或异常打断时的完整寄存器现场，包括通用寄存器、程序计数器、用户栈指针和处理器状态等。系统调用参数和返回值也位于这份现场中。用户任务的 TCB 持有一个可选的 `TrapFrame`，任务被再次选中时，内核从中恢复用户程序。

`TaskContext` 只保存内核任务切换时必须长期保留的寄存器：返回地址 `RA`、栈指针 `SP` 和被调用者保存寄存器。`switch_to` 在汇编中保存旧 `TaskContext`、装入新 `TaskContext`，最后通过恢复后的返回地址继续执行。其他临时寄存器按照函数调用约定处理，因此不需要重复保存在这里。

运行现场	主要内容	使用位置
<code>TrapFrame</code>	通用寄存器、PC、SP、处理器状态和系统调用寄存器。	用户任务进入内核时保存，返回用户态前恢复；也用于 <code>clone/fork</code> 构造子任务的初始现场。
<code>TaskContext</code>	<code>RA</code> 、 <code>SP</code> 和被调用者保存寄存器。	内核任务与当前 CPU 的调度器槽之间切换；每个内核任务拥有独立内核栈和一份 <code>TaskContext</code> 。
调度状态	<code>Running/Ready/Blocked/Stopped/Zombie</code> TID 和调度参数。	保存在 TCB 中，决定一份现场是否允许恢复，以及恢复后应进入哪条执行路径。

两种现场各自解决一个层次的问题：`TrapFrame` 负责跨越用户态与内核态边界，`TaskContext` 负责在两个内核执行流之间更换栈和继续位置。临时发生在内核中的 trap 仍会形成中断现场，但内核任务暂停后的长期恢复位置由 `TaskContext` 保存。

4.7.2 用户任务的 `trap` 进入与返回

调度器选中用户任务后，按照下面的顺序恢复它：

- 通过 `try_start_running` 取得 `running_cpu` 所有权，把任务写入本 CPU 的 `CURRENT_TASK`，并将状态从 `Ready` 改为 `Running`。
- 从 TCB 中取出 `TrapFrame`，激活任务自己的 `MemorySet`，使用户虚拟地址指向正确的物理页。
- 检查待处理信号。如果信号要求停止或终止任务，本次不进入用户态；如果需要调用用户信号处理函数，则先修改返回现场。
- 调整体系结构相关的特权级和中断状态，再通过 `run_user_task` 恢复寄存器并返回用户态。

用户程序执行系统调用、发生时钟中断或出现页故障时，处理器通过 `trap` 再次进入内核，并把当时的寄存器状态写回当前 `TrapFrame`。系统调用处理程序从中读取调用号和参数，再把返回值写回对应寄存器。普通系统调用完成后，程序计数器越过调用指令；`execve` 和 `rt_sigreturn` 已经建立了新的返回现场，不再使用普通的 PC 前进逻辑。

一次用户态运行结束后，内核先恢复内核页表，再把更新后的 `TrapFrame` 放回 TCB，清除本 CPU 的 `CURRENT_TASK` 并释放 `running_cpu`。任务仍可运行时执行 `Running -> Ready` 并通过本地路径重新入队；任务已经阻塞、停止或退出时则保留相应状态，不进入就绪队列。下次可能由同一 CPU 或另一个满足 `affinity` 的 CPU 选中该任务，用户程序会从保存的 PC 和寄存器状态继续，而不是从程序入口重新开始。

4.7.3 内核任务的 `TaskContext` 切换

创建内核任务时，WIL0S 会分配独立内核栈，把 `TaskContext.SP` 指向栈顶，把 `TaskContext.RA` 指向任务入口。第一次被调度时，恢复这两个寄存器就能从入口函数开始执行。

每个 CPU 都在 `SCHEDULER_SLOTS` 中保存一份调度器现场。切入内核任务时，`switch_to` 把当前 CPU 的 RA、SP 和保存寄存器写入对应槽，再装入任务现场。内核任务主动让出 CPU、进入阻塞或退出时，切换方向相反：先把当前执行位置和内核栈保存到任务的 `TaskContext`，再恢复同一 CPU 的调度器槽。不同 CPU 不共享正在使用的调度器栈和返回位置。

因此，已经运行过的内核任务再次被选中时，会从上一次 `switch_to` 返回的位置继续执行。内核任务不需要激活用户地址空间，也不依赖用户 `TrapFrame` 恢复；这既减少了切换内容，也避免了纯内核执行流误用用户页表。

4.7.4 时钟中断与时间片抢占

普通调度模式的默认时间片为 10 ms。设置下一次时钟时，内核还会比较睡眠超时和进程间隔定时器的最近 `deadline`；如果某个事件更早到期，就把时钟提前到该时刻，而不是固定等待完整时间片。

时钟中断到来后，内核先调用 `wake_expired_timers`，把已经到期的等待任务改为 `Ready`。随后触发当前 CPU 的抢占：当前任务的运行现场被保存，状态由 `Running` 改为 `Ready`，释放执行所有权并通过本地路径重新入队，调度器再按第 4.5 节介绍的 `affinity`、队列和优先级规则选择下一个任务。用户任务通过 `trap` 路径保存现场，内核任务通过 `TaskContext` 回到本 CPU 的调度器槽。

这里需要区分“产生调度机会”和“决定下一个任务”。时钟中断只负责结束当前时间片并进入调度过程；下一个运行的是普通任务、FIFO 实时任务还是 RR 实时任务，仍由调度策略和就绪队列决定。因此，抢占机制不会绕过第 4.5 节中的优先级选择。

前台评测驱动采用不同的节拍控制：默认时间片延长为 50 ms，减少静态 `libc` 启动过程中频繁往返测试驱动的开销。只有测试 `deadline` 到期，或当前为用户任务且用户就绪队列中还有其他任务时，时钟中断才交出 CPU；否则只重新设置下一次时钟、处理待决信号，并让当前任务继续运行。前台模式也不会从内核任务队列选择任务。

4.7.5 切换触发条件与一致性保证

除了时间片到期，任务还可能因为主动让出、等待事件或退出而离开 CPU。不同原因决定了现场是否保存、任务是否重新入队：

触发原因	状态变化	现场处理
时间片到期	<code>Running -> Ready</code>	保存当前现场并重新入队，以后从被中断的位置继续。
<code>sched_yield</code>	<code>Running -> Ready</code>	任务主动结束本次运行机会；保存现场后让调度器选择其他任务。
阻塞等待	<code>Running -> Blocked</code>	保存现场但不加入就绪队列；事件、超时或信号到达后改为 <code>Ready</code> 。
作业控制停止	<code>Running/Ready -> Stopped</code>	保留可恢复现场但不调度；收到 <code>SIGCONT</code> 后重新进入 <code>Ready</code> 。
退出或致命信号	<code>Running -> Zombie</code>	执行退出清理，不再恢复该现场；由父进程通过 <code>wait4/waitid</code> 回收。

`execve` 是一个特殊情况：它不会创建新的调度实体，也不是一次普通的任务切换。内核替换当前进程的地址空间，并把现有 `TrapFrame` 改为新程序的入口、用户栈和参数现场；返回用户态后，同一个 TID 直接开始执行新程序。

为了保证切换过程一致，Wll0S 遵循以下顺序：先保存现场并更新任务状态，再清除本 CPU 的 `CURRENT_TASK`、释放 `running_cpu`，最后进入调度器；恢复用户任务时先取得执行所有权并激活它的地址空间，用户态返回后立即恢复内核页表。调度入口还会过滤 `Blocked`、`Stopped` 和 `Zombie` 项，就绪队列按 TID 去重，避免同一任务被重复恢复。

当前实现已经具备 CPU-local 当前任务与调度器槽、共享 affinity-aware 就绪队列、单 idle CPU 通知、blocked-owner 定向唤醒和跨核地址空间失效机制。任务可以在多个在线 CPU 之间迁移，但仍缺少 Linux CFS 式负载度量、per-CPU run queue 和主动负载均衡；这些属于后续调度扩展，而不是当前 SMP 正确性成立的前提。

4.8 本章小结

Wll0S 以线程作为调度基本单位，并使用统一的 TaskControlBlock 表示用户线程和内核任务。多个用户线程通过 ThreadGroup 组成进程，共享地址空间、文件描述符表和信号处理动作，同时保留各自的 TID、寄存器现场和等待状态。这种组织方式使调度器能够统一选择执行实体，也能够表达 Linux 中线程私有状态与进程级共享资源的区别。

在任务生命周期方面，模块实现了 clone/fork、execve、exit/exit_group、wait4/waitid 和孤儿进程重托管等关键路径。不同的 clone 标志决定资源是共享还是复制；退出后的 Zombie 任务保留必要的退出信息，最终由父进程完成回收。由此，任务创建、映像替换、退出和回收形成了相互衔接的处理流程。

调度器使用用户任务和内核任务两类就绪队列，并支持普通任务轮转以及 FIFO、RR 实时调度策略。任务等待 I/O、定时器、futex、信号或子进程事件时会进入 Blocked，事件发生后再返回 Ready。wait_token 和就绪队列去重机制用于识别有效唤醒，避免过期事件或重复通知破坏任务状态。

用户任务通过 TrapFrame 保存用户态现场，内核任务通过 TaskContext 保存内核执行位置。时钟中断、主动让出、阻塞和退出分别触发相应的状态变化与上下文切换，使多个任务能够在多个在线 CPU 上并行推进。running_cpu、CPU-local CURRENT_TASK 和每 CPU 调度器槽共同保证执行现场的唯一所有权；前台评测驱动仍复用真实的任务对象和系统调用语义，但对时间片、任务选择和结束条件施加了额外控制。

总体来看，当前模块已经形成从任务创建、SMP 调度运行、阻塞唤醒到退出回收的完整执行闭环，并为 Wll0S 运行 BusyBox、glibc/musl 程序和 LTP 测试提供了基础。后续工作将集中在 per-CPU run queue、负载均衡、严格区分 FIFO/RR 语义以及更完整的 Linux 进程统计，而不是再把多核调度作为尚未实现的远期目标。

第 5 章 文件系统与 I/O 模块

5.1 模块概述

文件系统模块负责把用户程序看到的路径和文件描述符，转换为内核中的文件、目录、管道和设备对象。它既要从磁盘镜像中读取程序与动态库，也要允许 Shell、运行库和测试程序在运行期间创建临时文件，并提供接近 Linux/POSIX 的访问方式。

我们的设计目标不是复刻完整 Linux 文件系统，而是在结构清晰的前提下建立一条可扩展的 I/O 路径。系统调用层负责参数和用户内存检查，VFS 负责路径、权限和挂载视图，文件描述符层负责打开对象的状态与生命周期，最后由 MemFS、ext4、VFAT、管道或设备完成实际操作。这样的划分使不同后端能够共享上层接口，也便于根据评测结果逐步补充语义。

5.2 总体架构与核心对象

文件操作自上而下分为四层。第一层接收用户请求并检查访问范围；第二层维护统一的路径和挂载视图；第三层用文件描述符连接进程与已打开对象；第四层保存数据或提供具体 I/O 行为。各层只承担相对稳定的职责，避免路径处理、进程资源和磁盘读写相互混杂。

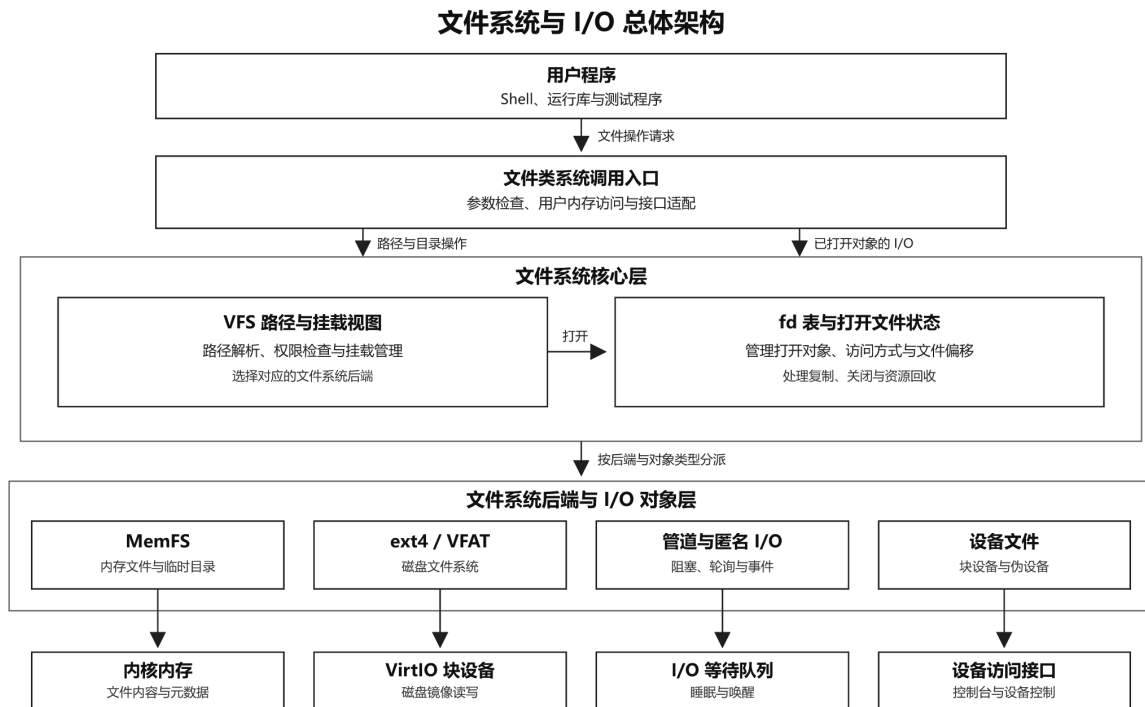


图 5-1 文件系统与 I/O 总体架构

VFS、文件描述符表和打开文件状态是这一架构中的三个核心对象。VFS 描述“路径指向什么”，文件描述符表描述“进程可以访问什么”，打开文件状态描述“本次打开如何访问”。普通文件、目录、管道和设备在行为上不同，但都能通过文件描述符成为进程资源。底层后端则保留各自特点：**MemFS** 面向运行期对象，**ext4** 面向磁盘数据，**VFAT** 提供只读挂载能力，管道和设备承担特殊 I/O。

核心对象	保存内容	主要作用
VFS 视图	路径、节点信息和挂载关系	统一路径语义，并选择实际文件系统后端。
文件描述符表	进程可见的 fd 与描述符标志	把整数句柄映射到已打开的内核对象。
打开文件状态	访问方式、文件偏移、对象引用与锁 owner	表达一次打开产生的访问属性，并记录当前实现中的共享边界。
文件系统后端	目录结构、文件数据与缓存	完成实际的创建、查询、读写和同步。

5.3 路径解析、权限与挂载管理

路径解析从进程可见的文件系统环境开始。绝对路径以进程根目录为起点，相对路径以当前工作目录或目录描述符为起点。内核在规范化路径时处理重复分隔符、当前目录和父目录，但不会允许解析结果越过进程根目录。随后逐级检查目录类型和搜索权限，并按规则跟随符号链接。这样可以在进入具体后端前统一处理常见路径错误。

权限检查与任务凭据结合进行。打开文件时检查目标节点的访问权限，创建或删除目录项时还要检查父目录的写权限和搜索权限。有效用户与组决定一般文件访问，文件系统身份用于实际的路径操作；对粘滞目录等特殊情况再增加所有者限制。当前模型覆盖比赛测试常用的所有者、用户组和其他用户权限，但尚未实现 **capability** 与 **POSIX ACL**。

挂载表把逻辑路径映射到实际后端。启动后的根视图由磁盘根文件系统和 **MemFS** 运行期目录共同组成，同名运行期对象可以覆盖镜像中的只读视图。额外挂载的 **VFAT** 卷保持独立边界；跨后端的硬链接和重命名不会隐式复制文件，而是返回跨设备错误。通过这一规则，路径视图保持统一，同时不会掩盖不同后端的能力差异。

5.4 命名空间事务与查找缓存

并发路径操作不能只保证单次 **ext4** API 调用安全，还要保证“解析父目录、修改目录项、更新 **inode**、失效缓存”作为一个整体对其他 CPU 可见。**Wll0s** 因此在 **ext4** 后端设置命名空间事务锁和底层 **mutation** 锁。普通 **lookup** 持有共享命名空间事务，创建、删除、链接和重命名则从第一次路径解析开始持有排他事务，直到目录与 **inode** 更新完成并撤销相关缓存。普通文件数据读写和回写不持命名空间事务锁，避免把大文件 I/O 与目录修改无关地串行化。

机制	保护范围	设计作用
NAMESPACE_LOCK	路径解析、目录项与缓存发布	使 reader 只接受同一事务与 generation 内的结果，使 writer 的目录变化原子可见。
EXT4_MUTATION_LOCK	bitmap、inode 和底层 ext4 transaction	补足 ext4 后端内部串行化边界，避免并发 writer 重复分配底层资源。
路径与目录缓存	positive、negative、目录快照和 inode metadata	writer 精确失效受影响路径并推进 generation，reader 不发布跨代结果。
VFS 解析缓存	父目录符号链接与 readlink 结果	使用 64K 有界容量和逐项淘汰，避免大型编译工作集到达上限时整表变冷。

open_path 完成 tmpfs 与 ext4 后端选择后，对同一 ext4 路径只执行一次 generation-validated lookup_kind，取得可复制的 (inode, kind)，并在本次打开流程中的目录、普通文件和符号链接判断之间复用。这个结果只在一次调用生命周期内使用，不拥有 inode、打开文件对象或页缓存；rename、unlink 和 symlink mutation 仍通过原有失效协议撤销相关缓存。因此，查找复用减少的是重复路径遍历，不改变后端选择和命名空间可见性。

5.5 文件描述符与资源生命周期

文件描述符是用户程序访问内核 I/O 对象的统一句柄。每个进程通过 fd 表保存普通文件、目录、管道和设备等对象，并为每个表项记录执行程序时是否自动关闭。访问模式、追加和非阻塞等状态由打开对象保存，进程资源上限则约束可分配的描述符数量。

描述符复制时，新 fd 与原 fd 会继续引用相同的文件 backing、管道端点或 socket/eventfd 状态；普通文件还共享一个 OpenFileDescriptionOwner，用于维护 OFD 锁和 lease 的生命周期。当前 FileDescriptor 仍按值保存普通文件偏移、追加和访问属性，复制表项时这些字段形成快照，因此尚未完整实现 Linux 中由同一 open-file description 共享偏移和 status flags 的全部语义。创建子进程时，内核可以共享整张 fd 表，也可以复制表项并保留上述底层引用；工作目录、根目录和文件创建掩码则由另一组文件系统上下文管理。

阶段	资源变化	设计意义
打开	创建内核对象并分配最低可用 fd	建立用户句柄与实际对象的联系。
复制	复制表项或共享整张 fd 表	支持重定向、管道和进程创建。
执行替换	关闭带有执行时关闭标志的表项	避免新程序继承不需要的资源。
关闭	移除表项并减少对象引用	最后一个引用离开时完成实际回收。

资源回收以引用关系为基础。`ext4` 普通文件从目录中删除后，如果仍有打开引用，内核先移除目录项并把 `inode` 放入延迟删除集合，已有描述符可以继续访问内容；最后一个引用关闭后才执行截断、回收 `inode` 和缓存清理。关闭管道端点时，内核同时更新读写端数量并唤醒对端；进程退出时则统一释放整张 `fd` 表。`OpenFileDescriptionOwner` 的最后一个引用销毁时还会释放对应 `OFD` 锁，生命周期处理因此集中在对象引用边界，而不是分散到每个系统调用。

5.6 文件读写与偏移维护

普通读写以当前 `FileDescriptor` 保存的偏移为起点。操作成功后，顺序读写推进该描述符副本的偏移；追加写在每次写入前重新取得文件末尾，以避免沿用旧位置。固定偏移读写直接使用调用者给出的起点，不改变描述符偏移，适合多个任务对同一文件的不同区域进行访问。由于完整共享 `OFD` 偏移仍未实现，当前语义应明确描述为“同一 `fd` 表项共享、复制表项保存偏移快照”，不能把所有 `dup/fork` 场景写成已经与 `Linux` 完全一致。目录和管道不具备普通文件的定位能力，因此会使用各自的读取规则。

向量 `I/O` 把多个用户缓冲区组成一次逻辑操作。内核先分批读取缓冲区描述，检查地址和总长度，再按顺序完成数据传输。较小的数据使用临时缓冲，较大的请求分块处理，从而限制内核栈和临时内存开销。如果前一部分已经完成、后一部分发生错误，则返回已传输的字节数。这一处理方式既保留了常见 `POSIX` 语义，也适合 `iozone` 等连续读写场景。

所有用户数据都在系统调用边界完成检查和复制，后端只接收内核可以安全访问的缓冲区。路径解析和存储后端因此不需要直接理解用户地址空间，文件系统与内存管理之间的边界也更加明确。

5.7 MemFS、ext4 与缓存同步

`MemFS` 和 `ext4` 共同构成系统的主要文件视图。`MemFS` 保存临时文件、运行期目录和伪设备节点，访问速度快，但重启后数据不会保留。`ext4` 位于 `VirtIO` 块设备之上，保存用户程序、动态库、测试文件和需要持久化的数据。`VFS` 在两者之间统一路径和元数据语义，上层读写不需要关心数据最终位于内存还是磁盘。

为了控制大文件占用，`MemFS` 对不超过 `1 MiB` 的文件采用连续存储，对更大的文件改用 `64 KiB` 分块表示。全零的数据块可以只记录为空洞，读取时再补零。这种策略降低了测试生成大段空数据时的内存压力，同时保持普通读写、截断和固定偏移访问看到一致的文件内容。

`ext4` 数据面采用“小文件 `dense`、大文件 `sparse`”的分层缓存。文件不超过 `8 MiB` 时可以保留连续字节数组；更大的文件只在实际修改页面时，以 `BTreeMap<page_index, FrameTracker>` 保存页缓存，尚未缓存的读取直接进入 `extent-aware read_at`。这种设计不会为了少量写入把整个大型编译产物复制进连续内核缓冲，也能正确读取多层 `extent tree` 后部的数据块。


```
struct CachedRegularFile {
    dense: Option<Vec<u8>>, pages: BTreeMap<usize, FrameTracker>,
    size: usize, dirty: PageCacheDirtyModel,
}
```

代码 5-1 ext4 普通文件缓存的关键结构

写入先修改 dense 或 sparse 缓存，并记录 dirty range。回写把脏区间切分为最多 256 KiB 的 cluster，对连续物理块进行批量写入；未对齐的 partial block 仍执行 read-modify-write。truncate 会清理新 EOF 所在块的尾部字节，避免文件再次扩展时暴露旧内容。clean-page cache、physical-block-run cache、inode metadata cache 和 executable image cache 都有明确容量，并由 namespace 或 inode mutation 触发对应失效。

写回时先在锁内取得稳定状态，再在锁外完成耗时块 I/O，最后确认期间是否又产生新修改。这一设计缩短了全局锁持有时间，也避免一次写回覆盖并发写入。当前系统主要依靠显式同步、最后引用关闭和机会式 writeback queue 推进落盘；后台 writeback worker 默认关闭，仍属于需要继续验证的优化路径。

5.8 管道、等待与设备 I/O

管道使用内核缓冲区连接读端和写端。缓冲区为空且写端仍存在时，阻塞读进入 I/O 等待队列；缓冲区写入数据后再唤醒读者。缓冲区已满时，写者采用相同方式等待空间。非阻塞描述符不进入睡眠，而是立即返回当前状态；全部写端关闭后，读端获得文件结束，全部读端关闭后，写入失败并产生相应信号。

轮询接口采用“检查状态、登记等待、再次检查、唤醒后重扫”的流程。登记后的第二次检查用于缩小状态变化与任务睡眠之间的时间窗口，避免丢失唤醒。该机制复用了第四章介绍的阻塞与唤醒模型，使文件系统只负责判断对象是否可读、可写，调度器负责真正的睡眠和恢复。

设备 I/O 也通过文件描述符接入。VirtIO 块设备为 ext4 和 VFAT 提供统一的块访问能力，loop 设备允许普通文件临时作为块设备使用；空设备、零设备和随机设备提供常用的字符设备行为。当前设备层以比赛运行环境为重点，尚未建立完整的通用驱动注册框架，终端控制等复杂语义也只提供有限支持。

5.9 测试验证与当前边界

文件系统验证采用“基础功能、接口语义、综合负载”三个层次。基础功能关注创建、打开、读写、目录遍历和删除；接口语义通过 LTP 检查权限、符号链接、描述符状态和错误码；综合负载则使用 BusyBox、运行库程序、iozone 和 BuildStorm 检查较长执行链、大文件读写与并发命名空间变化。串口日志用于确认真实内核运行结果，测试脚本和源码检查只作为补充证据，不能替代端到端通过记录。

验证层次	关注内容	使用方式
基础路径	文件、目录、挂载和设备的基本访问	通过内核启动后的独立用例检查。

验证层次	关注内容	使用方式
兼容语义	权限、链接、fd 状态、阻塞与错误码	使用有界 LTP 用例逐项验证。
综合负载	大文件、连续读写和用户程序 workflow	结合 iotest、BusyBox、运行库程序与完整编译观察。
SMP 生命周期	并发 create/unlink/rename、open-unlink 与缓存失效	在双架构多 CPU 回归中检查 generation、inode 生命周期和数据路径。

当前实现已经形成从用户请求到 MemFS、ext4、VFAT、管道和设备的完整基本路径。双架构 SMP 生命周期回归覆盖并发 namespace create/unlink/rename、open-unlink、regular-file、缓存失效和大文件压力；完整 BuildStorm 也已经越过旧的 rustc metadata ENOENT 边界，验证了命名空间事务和数据缓存能够支撑真实编译 workflow。但它与完整 Linux 仍有差距：VFAT 目前只读，跨后端链接和重命名受限，tmpfs 尚未形成相互隔离的独立实例；完整共享 OFD 偏移、capability、ACL、挂载命名空间和终端语义尚未实现。后台写回、复杂轮询组合以及更多多进程大文件压力仍需要持续验证。

5.10 本章小结

本章介绍了 wlllos 文件系统与 I/O 模块的整体设计。系统以 VFS 统一路径、权限和挂载视图，以命名空间事务保证并发目录变化的原子可见性，以文件描述符组织进程资源，再由 MemFS、ext4、VFAT、管道和设备完成具体操作。generation-validated lookup、分层文件缓存、等待队列与引用生命周期分别保证路径一致性、数据访问效率、阻塞协作和 open-unlink 语义。当前实现以比赛环境中的真实程序和有界兼容测试为重点，后续将继续完善共享 OFD、独立挂载、后台写回、权限模型和设备语义。

第 6 章 系统调用与兼容模块

6.1 系统调用总体设计

系统调用是用户程序进入内核的统一边界，也是 Linux 兼容语义汇合的位置。wll0s 在这一层完成调用号识别、参数解释、用户内存访问、内核对象操作和错误返回，但不在分发入口堆叠具体业务。进程、地址空间、文件描述符、信号和套接字仍由各自模块维护，系统调用层只负责把 ABI 请求转换为这些对象能够理解的操作。

设计目标包括三点：一是让同一用户态接口在 RISC-V64 与 LoongArch64 上保持一致语义；二是把用户地址与内核地址严格分开，避免直接解引用不可信指针；三是让阻塞、信号中断和错误码具有可观察且可恢复的行为。系统调用覆盖以真实程序所需的对象语义为依据，不以固定返回成功代替尚未实现的能力。

体系结构入口	提取调用号与参数，保存用户现场并回填返回值
统一分发层	按 Linux ABI 分类请求，统一成功值与错误码
边界转换层	检查用户地址，复制字符串、结构体和跨页缓冲区
对象语义层	操作任务、地址空间、文件、信号、时间和网络对象
等待与调度层	登记等待条件、超时和唤醒结果，必要时让出处理器

图 6-1 系统调用分层架构

模块按照内核对象而不是调用号数量组织。文件与描述符接口进入 VFS 和描述符表，进程接口进入任务与线程组，内存接口进入进程地址空间，信号与同步接口进入任务状态和等待队列，网络接口复用描述符生命周期。这样的边界使新增接口能够复用既有对象规则，也避免同一种权限、等待或回收语义在多个入口中重复实现。

6.2 调用入口与分发流程

用户态执行陷入指令后，体系结构入口先从约定寄存器中取得调用号和最多六个参数，并保存当前用户现场。统一分发层只进行轻量的类别选择和类型转换，再把请求交给相应对象处理。完成后，成功结果写回返回寄存器，失败结果转换为 Linux 约定的负错误码，用户程序无需感知内核内部模块划分。

普通调用在返回前推进用户程序计数器，并处理已经到期的定时事件。程序映像替换和信号现场恢复会直接构造新的用户现场，因此不能再套用普通返回步骤。任务退出同样不会回到原调用点。入口层显式区分这些情况，避免新入口地址被旧的系统调用收尾逻辑覆盖。

未识别的调用号统一返回 `ENOSYS`，已经识别但参数或状态不合法的调用则返回与具体语义对应的错误。分发层不伪造对象，也不吞掉失败信息。调试日志可以记录调用号和边界状态，但正式运行路径只保留必要信息，避免高频系统调用因日志输出形成额外串行开销。

6.3 用户内存访问

用户指针只表示用户地址空间中的位置，内核不会把它直接当作可信指针使用。路径、参数数组、结构体和 I/O 缓冲区都通过统一复制边界进入或离开内核。复制前先检查空指针、地址溢出、用户范围和访问方向，再以页为单位完成地址转换，因此跨页对象不会因为只验证首地址而越过无效区域。

读方向要求源区域可读，写方向要求目标区域可写。检查过程与内存管理的按需分配和写时复制规则相连：合法但尚未驻留的页面可以在复制前补齐，写入共享的只读副本时先完成私有化；真正越界或权限不符的访问才返回 `EFAULT`。这样，系统调用层不需要分别理解匿名页、文件页和写时复制页的内部差异。

字符串读取设置统一的最大长度，并区分普通字符串、路径和允许空值的特殊参数。固定结构体按照用户 ABI 布局整体转换，向量 I/O 则先验证向量数组，再逐段处理实际缓冲区。部分完成是否允许由具体接口决定，底层复制只报告准确的边界结果，从而保持文件、信号、时间和网络接口的一致性。

进入内核后使用稳定的内核页表执行，用户数据复制通过当前任务的地址空间完成显式转换。任务切换、异常嵌套和多核运行都遵守这一规则，防止把一个进程的用户地址解释到另一个地址空间，也避免设备驱动或文件系统在持锁期间意外访问用户页。

6.4 内核对象与资源协同

系统调用的核心不是编号本身，而是编号背后的内核对象。进程创建会组合地址空间、文件表、信号处理方式和线程组关系；程序装载会替换地址空间并重建用户现场，同时清理带有执行时关闭标记的描述符；进程退出则需要回收映射、解除共享内存挂接、清理等待项并通知父进程。跨模块操作按“先构造新状态，再提交可见状态，最后释放旧资源”的顺序执行，避免失败路径留下半更新对象。

文件、管道、事件对象和本地套接字统一放入描述符表。系统调用只根据描述符取得对象及访问方式，实际读写、轮询和关闭行为由对象后端决定。这样可以让普通读写、向量读写、固定偏移读写和就绪等待复用同一生命周期，同时让关闭、复制和程序装载时的描述符处理保持一致。

接口族	核心对象	主要协同关系
进程与线程	任务、线程组、用户现场	地址空间、文件表、信号状态和父子关系共同参与生命周期。
内存	地址空间与虚拟区域	与页表、文件后端、共享段及缺页处理协同。
文件与事件	描述符表和文件对象	统一普通文件、管道、轮询对象与设备的访问和回收。
信号与同步	信号状态和等待队列	结合任务阻塞、超时、唤醒和退出清理。
网络	套接字描述符	复用描述符标志、就绪查询和本地数据队列。

内存、文件和任务之间也通过稳定边界协作。文件映射只向地址空间提供数据来源和写回能力，地址空间负责区域权限与驻留页；用户数据复制只调用地址空间检查，不直接

修改页表；等待队列只记录任务、条件和结果，不拥有文件或套接字对象。对象所有权清晰后，失败回滚和进程退出可以沿相同路径释放资源。

6.5 阻塞调用与可恢复执行

管道读写、子进程等待、轮询、睡眠、信号等待和 `futex` 都可能暂时无法完成。`wll0s` 将这些接口收敛到统一等待模型：先登记等待对象、原因、条件和可选截止时间，再次确认条件仍未满足后才阻塞任务。事件在登记与复查之间发生时，调用可以直接继续，从而避免经典的“检查后睡眠”丢失唤醒问题。

每次等待分配新的标识，唤醒者只有匹配当前标识才能改变任务状态。旧定时器、过期事件或重复唤醒即使稍后到达，也不会错误唤醒任务的新一轮等待。唤醒结果区分条件满足、超时和信号中断，系统调用据此返回正常结果、超时错误或 `EINTR`，而不是把不同原因压缩为同一个成功值。

当前前台执行模型允许阻塞的系统调用暂时停放，并在任务重新获得处理器后从原调用点恢复。恢复时先消费已经保存的等待结果，再清理等待队列和定时器登记，避免重复入队或重复执行具有副作用的步骤。进程创建、固定偏移写入和资源分配等接口把不可重复阶段放在阻塞点之外，保证恢复路径具有幂等边界。

任务退出或被终止时，内核会从就绪队列、I/O 等待、子进程等待、定时器和 `futex` 队列中移除相关登记。资源对象不会依赖已经失效的任务指针，父进程和同步等待者也能收到明确事件。该清理顺序是系统调用生命周期与调度器协作的重要组成部分。

6.6 信号、同步与时间语义

信号同时影响用户现场、任务状态和阻塞调用。每个任务保存屏蔽集合与待处理状态，线程组共享处理方式。投递时先判断目标与权限，再记录待处理信号；进入用户态前，内核选择未屏蔽信号，构造用户信号帧并切换到处理入口。信号返回从用户帧恢复寄存器、浮点状态和屏蔽集合，因此不同体系结构的现场布局由架构适配层分别处理。

默认终止信号会结束相应任务或线程组，等待中的任务则以中断结果被唤醒。临时信号掩码、定时等待和子进程退出通知复用统一等待模型，使信号到达、超时和正常事件之间具有确定顺序。当前实现覆盖常用处理、屏蔽、等待、返回和定向投递语义，高级实时信号排队、备用信号栈和完整调用重启仍作为后续边界。

`futex` 以用户地址中的整数为快速路径，只有条件不满足时进入内核等待。等待前验证地址对齐并比较当前值，再以地址空间或共享对象生成等待键；唤醒只匹配相同键和位集合的任务。线程退出时还会处理清除子线程标识和基础 `robust list` 清理，降低互斥量永久遗留为占用状态的风险。优先级继承和更复杂的重排操作尚未完整实现。

时间接口统一使用内核时钟和定时器队列。读取时间直接生成用户 ABI 结构，睡眠和带超时等待登记绝对截止时间，周期定时器到期后重新装载并投递相应信号。当前时间精度能够支撑常用运行时与基准程序，但虚拟时间、性能时间和完整资源统计仍采用简化模型。

6.7 错误返回与兼容策略

系统调用兼容不仅体现在成功路径，还体现在失败原因能否被用户程序准确区分。wll0s 在最接近错误来源的层次产生错误码：用户复制报告地址错误，地址空间报告映射或内存不足，描述符表报告编号和访问方式错误，VFS 报告路径、类型和权限错误，等待层报告超时或信号中断。上层只传递或进行必要转换，避免不同接口对同一问题给出互相冲突的结果。

错误类别	典型返回	判定原则
接口不存在	ENOSYS	调用号尚未进入分发表，不与参数错误混用。
用户地址无效	EFAULT	地址越界、权限不足或复制目标不可访问。
参数组合非法	EINVAL	调用已经存在，但标志、长度或状态组合无意义。
对象或路径错误	EBADF、ENOENT	区分无效描述符、对象不存在与路径解析失败。
权限受限	EACCES、EPERM	分别表达访问检查失败与操作资格不足。
资源暂不可用	EAGAIN、ENOMEM	区分可重试状态和无法完成的资源分配。
等待被打断	EINTR、ETIMEDOUT	保留信号中断与截止时间到达的差别。

对已经识别但仅实现部分语义的接口，内核只在明确安全的范围内提供兼容行为。提示性请求可以忽略而返回成功，改变对象状态或影响安全边界的未知选项则应拒绝。固定成功仅用于没有可观察副作用的兼容项，并在实现中保持集中；其余能力通过真实对象状态或明确错误暴露当前边界。

错误路径与成功路径使用相同的回滚规则。用户输出只在核心操作成功后提交，临时分配对象由所有权机制自动回收，已经登记的等待与定时器在返回前撤销。这样既便于用户程序采取恢复策略，也使比赛现场可以根据错误码快速判断问题位于 ABI、对象状态还是资源压力。

6.8 跨架构 ABI 适配

RISC-V64 与 LoongArch64 对外都采用 Linux 风格系统调用语义，但陷入指令、调用号寄存器、用户现场布局和部分参数位置并不相同。wll0s 将这些差异限制在体系结构入口、现场构造和少量参数适配处，统一分发层仍接收相同的调用号与六参数数组。对象实现不直接判断寄存器编号，因而绝大多数兼容逻辑可以在两种架构之间共享。

进程创建需要特别处理线程局部存储与子线程标识参数的位置差异，信号交付需要分别保存和恢复两种架构的整数、浮点及控制状态，程序装载则要按照目标 ABI 构造入口寄存器和用户栈。平台时钟频率、页表切换和异常原因也由硬件抽象层转换为统一语义。

跨架构适配遵循“边界转换一次，核心逻辑保持一致”的原则。新增系统调用时，先确认是否只涉及通用对象；只有寄存器布局、结构体 ABI 或硬件行为确有差异时才增

加架构分支。这样可以减少双架构实现漂移，也便于用同一用户程序对比返回值和对象状态。

6.9 兼容能力与扩展边界

当前系统调用层已经覆盖进程创建与映像替换、用户地址空间、常用路径和描述符 I/O、身份信息、基础信号与 `futex`、时间、轮询以及本地套接字等运行路径。它们均连接真实任务、页表、文件或等待对象，能够支撑 `shell`、动态链接程序、常用 C 运行库和现有基准负载。

兼容性仍有明确边界。进程组与作业控制、完整线程取消、实时信号队列、优先级继承 `futex`、文件映射脏页跟踪和精确资源统计尚未覆盖全部 Linux 语义。文件同步可以完成当前后端的数据提交，但完整日志与持久化保证仍受文件系统实现限制；套接字目前以本地内核队列为主，尚未形成面向外部网络的完整协议栈。

后续扩展按照内核对象推进，而不是孤立增加调用号。新增接口应先确认对象状态、并发规则、失败回滚和退出清理，再接入用户 ABI；已有接口则优先提高语义精度和多核一致性。对现场调试而言，首先根据错误码确定失败层次，再检查用户边界、对象生命周期和等待状态，可以比逐个追踪分发分支更快定位问题。

6.10 本章小结

wlllos 以统一陷入入口和 Linux ABI 分发为起点，通过用户内存边界连接任务、地址空间、文件、信号、时间与网络对象。阻塞调用使用可恢复等待状态，错误码在最接近原因的位置产生，体系结构差异集中在寄存器与用户现场适配层。该设计使系统调用既能服务当前用户程序，也能在不破坏既有对象边界的前提下继续扩展兼容能力。

第 7 章 总结与展望

7.1 项目总体总结

wll0s 是面向操作系统内核赛道构建的 Rust 类 Unix 内核。项目以真实用户程序为目标，在同一套内核主体上支持 RISC-V64 与 LoongArch64，并形成了从启动、内存管理、进程调度、文件系统到 Linux 系统调用的完整运行链路。内核不以单个测例的表面通过为最终目标，而是尽量让用户程序观察到的结果来自真实任务、页表、文件和等待对象。

系统整体采用分层与对象化设计。硬件差异集中在启动、中断、页表和上下文接口；内存管理维护虚拟区域、驻留页面及其所有权；任务系统统一组织进程、线程、调度和等待状态；VFS 与描述符层连接 MemFS、ext4、管道、设备和本地套接字；系统调用层负责将 Linux ABI 请求转换为各对象的操作。模块之间通过稳定边界协作，减少了跨架构重复实现和局部修复带来的耦合。

经过持续迭代，内核已经能够支撑动态用户程序、常用命令环境、文件与进程负载、多线程同步以及多核编译任务。项目关注的不只是功能数量，还包括地址空间一致性、任务退出回收、文件命名空间并发、用户指针安全和错误码准确性。这些语义共同决定复杂工作负载能否长时间稳定运行。

项目同时建立了面向比赛的工程闭环。功能实现、性能优化和错误定位均以源码、串口日志、自动解析结果和独立回归为依据；诊断构建与正式构建相互隔离，调试信息不改变系统调用结果、调度策略和评测输出。由此形成的内核既可以参加自动评测，也便于在现场根据模块边界快速判断问题来源。

7.2 初赛与决赛阶段总结

初赛阶段的重点是建立可运行的内核基础。团队完成了双架构启动、用户程序装载、基本地址空间、进程创建与等待、内存文件系统、文件描述符以及常用系统调用，使内核具备运行基础测试、命令程序和脚本的能力。这一阶段首先解决“能否运行”的问题，并逐步形成了内存、任务、文件和系统调用的模块边界。

随着测例范围扩大，初赛实现也暴露出教学型内核常见的不足：资源共享关系不够完整，错误路径与退出清理分散，部分文件和等待语义只适用于短任务。团队没有继续在分发入口叠加特殊判断，而是把问题下沉到地址空间、线程组、描述符表和等待队列中处理，为决赛阶段运行更复杂的负载奠定基础。

决赛阶段以真实多核编译任务为主要压力场景。内核进一步完善线程共享、写时复制、futex、信号、ext4 写入、文件命名空间事务、跨核唤醒和页表失效，并针对大内存、多核启动、VFS 查询以及高频内存分配进行优化。开发方法也从逐项补齐接口转向定位全局瓶颈：先通过可关闭的诊断统计建立根因模型，再实施通用修改，最后以双架构独立回归和正式镜像运行验证。

阶段	建设重点	形成的能力
初赛	启动、用户程序、基础内存、进程、MemFS 与常用系统调用	建立双架构最小运行闭环，能够执行基础测试和命令程序。
过渡阶段	共享资源、错误语义、动态装载、ext4 与阻塞等待	由单测可运行转向复杂程序可持续运行。
决赛	SMP、一致性、真实文件负载、性能诊断与 BuildStorm	形成双架构多核完整编译能力和可复现的验证链路。

在决赛最终验证中，RISC-V64 与 LoongArch64 均使用 QEMU 11.0.3、未修改的官方 `final-2026` glibc 镜像以及 8 GiB、8 vCPU 配置完成 clean build，并得到官方完整编译成功标记。当前正式候选的自动 judge 项在两种架构上均为 180/180；设计文档人工评审不包含在该自动结果中。所有结论均保留了内核与镜像哈希、运行参数、原始串口日志和解析输出，避免把环境缺失或脚本行为误记为内核能力。

从初赛到决赛，项目最大的变化不是系统调用数量增加，而是形成了面向复杂工作负载的整体架构。基础功能成为可复用对象语义，多核问题被纳入生命周期和一致性设计，性能优化也建立在可证伪的观测与对照实验之上。这一演进使内核从能够完成短测试，推进到能够承受长时间、多进程和多线程负载。

7.3 项目亮点与核心优势

wll0s 的亮点首先体现在“双架构共用核心”上。RISC-V64 与 LoongArch64 共享内存、任务、文件系统和系统调用主体，架构差异集中在硬件边界。双架构不仅能够启动相同类型的用户程序，也通过了相同的多核生命周期回归和官方编译任务，证明抽象层能够承载真实负载，而不只是完成编译适配。

第二个亮点是真实对象语义。进程创建、文件读写、内存映射、信号和同步操作均连接实际内核状态，失败路径返回可解释错误，任务退出会清理共享资源和等待关系。复杂测例暴露的问题通常被修复为模块级规则，而不是写成针对程序名称、路径或输出的特殊分支，因此新增负载可以继续复用已有能力。

第三个亮点是多核一致性与性能优化并重。地址空间标识、跨核 TLB 失效、可恢复阻塞、每 CPU 小对象缓存、驻留页分离管理和 VFS 查询复用分别作用于切换、等待、分配和文件访问热点。优化始终保留正确性门槛，并通过正式构建与诊断构建、生产对照与独立压力测试交叉验证。

核心优势	设计体现	比赛价值
双架构统一	共用内核对象模型，硬件差异集中在架构层	减少重复实现，便于同步修复和交叉验证。
真实 Linux 语义	以任务、地址空间、VFS 和等待状态支撑用户 ABI	结果可解释，能够承载相邻测例和真实程序。
多核可扩展	明确跨核发布、失效、唤醒和资源回收顺序	适应并行编译和长时间 SMP 压力。
存储路径完整	MemFS 快速路径与 ext4 持久化路径接入统一 VFS	兼顾启动效率、功能测例和文件型负载。
证据可复现	保存命令、版本、哈希、原始日志和解析结果	便于评委核验，也便于现场复跑和回归定位。

这些优势共同构成项目继续演进的基础。后续可以在保持现有对象边界的前提下，完善实板启动与设备支持、外部网络协议、文件持久化细节以及更完整的 Linux 线程和信号语义。对于团队而言，本次比赛积累的不只是一个可运行内核，也包括从现象复现、根因分析到双架构验证和证据归档的一套系统工程方法。

第 8 章 AI 辅助使用说明

8.1 使用原则

本项目在开发、调试和文档整理过程中使用了 OpenAI Codex / GPT-5 以及 VS Code 中的 AI Agent 工作流。AI 的定位是工程辅助工具，主要帮助团队阅读代码、整理问题、提出候选方案和组织验证材料，不替代参赛队员对内核架构、代码修改和评测结论的判断。

AI 输出只作为待验证输入。涉及系统调用语义、资源生命周期、并发正确性、性能提升和比赛成绩的内容，必须能够由当前源码、实际构建、原始串口日志或官方解析结果独立确认。无法找到工程证据的建议不进入正式实现，无法重复得到的结果不写入技术结论。

团队对 AI 建议采用“理解上下文、缩小范围、实施验证、保留证据”的使用方式。先向工具提供必要的代码和现象，再把建议转换为可证伪的问题；候选修改完成后，通过双架构构建、相关回归和正式工作负载判断是否保留。该流程既利用工具提高信息整理效率，也保持技术决策的可追溯性。

8.2 使用场景与验证方式

AI 首先用于理解和审查工程上下文。开发过程中会结合源码、变更差异、评测脚本和历史日志梳理对象关系，检查用户指针、锁顺序、资源回收和跨架构分支等风险。工具给出的解释需要与实际调用路径逐项对照，不能以概括性回答代替源码阅读。

在问题定位中，AI 用于把日志现象整理为可证伪的根因模型，并列出需要区分的层次。例如 BuildStorm 阶段曾围绕地址空间、调度、VFS 和文件命名空间建立候选排序，再通过可关闭的诊断统计、生产对照和独立 SMP 回归排除错误假设。最终结论由实验结果决定，而不是由模型推断本身决定。

使用环节	AI 辅助内容	人工验证方式
源码理解	梳理模块关系、变更差异和 Linux ABI 边界	阅读实际实现，核对对象状态和调用路径。
问题诊断	汇总日志，提出根因模型、反例和候选优先级	使用聚焦复现、诊断统计和对照实验逐项证伪。
实现与审查	辅助形成通用修改，检查锁、所有权、错误码和退出清理	队员审阅补丁，并执行构建、回归和压力测试。
实验组织	整理双架构命令、运行顺序、日志解析和哈希留存	核对实际参数、进程状态、原始串口和官方 judge 输出。
文档整理	组织章节，压缩重复描述并统一术语	逐项确认描述与源码、支持范围和证据等级一致。

AI 还用于辅助组织双架构 QEMU 运行与证据归档，包括检查工作区状态、记录构建参数、汇总原始日志和整理复现步骤。运行仍由真实脚本和内核完成，AI 只协助控制过程

与核对结果。每项正式结论都能够脱离对话记录，从仓库、命令和保存的产物中重新验证。

8.3 工程边界与责任

AI 没有修改官方镜像、测试套件、`guest` 脚本、`judge`、评测标记或 `guest` 时间，也没有根据测试名称、编译目标、路径、命令或输出改变正式内核行为。诊断功能只在独立构建选项下启用，正式候选默认关闭；评测结果来自真实用户程序、官方脚本和未修改的解析流程。

AI 不拥有项目仓库和比赛提交的最终决定权。代码是否采纳、实验是否启动、结果如何解释以及材料是否提交，均由参赛队员确认。团队负责检查 AI 输出中的幻觉、遗漏和过度推断，并对所有进入仓库的修改、文档表述和成绩声明承担责任。

因此，本项目形成的使用边界可以概括为：**AI 辅助理解与执行**，队员负责设计与决策；**AI 提供候选解释**，实验负责判定真伪；**AI 帮助整理证据**，原始产物作为最终依据。后续开发仍将遵循“AI 辅助、人工判断、实测确认、证据归档”的流程，使工具效率服务于真实工程质量。